ЛЦТ-2026 · СПЕЦИАЛЬНЫЙ ТРЕК ФСП

ФСП Талант

Платформа подбора ИТ-специалистов с обратной механикой, адаптивным тестированием на грейд и верифицированным профилем достижений ФСП. Сопроводительная документация.

Краткое описание подхода (промежуточная сдача)

ФСП Талант — веб-платформа подбора ИТ-специалистов с обратной механикой: категорию кандидата определяет адаптивный тест, а работодатель сам выходит на кандидата с предложением и зарплатной вилкой.

Механика тестирования

1. **Опрос:** отрасль, специализация, стек, три вопроса самооценки, предполагаемый грейд.
2. **Адаптивный тест (IRT 3PL + CAT)** на заявленный грейд, 10–18 заданий. Следующее задание выбирается по максимуму информации с балансом навыков по blueprint; при тесте по вакансии blueprint строится из её навыков.
3. **Задания — параметрические семейства:** 59 семейств, 150 калиброванных позиций. Структура и сложность фиксированы, данные генерируются. Ответ вычисляется исполнением кода, SQL-движком или точной моделью семантики, LLM для этого не нужна. У каждого кандидата свой экземпляр, поэтому слитый ключ бесполезен.
4. **Грейд** присваивается при $P(\theta \geq \text{порог}) \geq 0{,}6$.
5. Не подтвердился — без понижения, сразу доступен тест на ступень ниже.
6. Подтвердился уверенно — 7 дней доступен тест на ступень выше.
7. Смена грейда — не чаще раза в 90 дней.
8. Неподтверждённый грейд не прячет кандидата: он виден работодателю ниже подтверждённых, с пометкой.
9. **Защита от утечек:** параметризация, контроль экспозиции (Sympson–Hetter по специализации) с лимитом семейства, CUSUM-детектор утечек по позициям, сигналы целостности без автоматических штрафов.
10. **Регулярные задания** от работодателей раз в неделю, оценка по прозрачной рубрике, индекс актуальности профиля.

Механика подбора

1. Работодатель описывает потребность текстом. NLP извлекает специализацию, грейд, обязательный и желательный стек, формат, город и вилку.
2. Система рекомендует категории — целевую и соседние — и ранжирует кандидатов объяснимым скорингом:
3. стек, подтверждённый тестом, весит больше заявленного;
4. учитываются грейд, сила результата теста, ФСП, зарплата, формат и актуальность.

У каждого кандидата есть «чипы» с плюсами и минусами. 3. Фильтры уточняют подборку без её потери, закреплённые кандидаты остаются в выдаче. 4. Приглашение: - анонимная карточка до согласия кандидата; - перед отправкой — проверка «вилка попадает в ожидания» без раскрытия суммы; - вилка обязательна; - квота 20 приглашений в неделю. 5. Принятие открывает контакты. Отказ — с причиной, и веса ранжирования компании адаптируются. 6. Дополнительно: классический отклик на вакансию, траст-скор работодателя, вебхуки для ATS.

ФСП ID и достижения

- Привязка через OIDC (Keycloak, realm `fsp`) и сервис реестра по предложенному нами контракту.
- Сигнал ФСП = результат × давность по каждому соревнованию плюс разряд. Иерархии соревнований и дисциплин нет, участия без места ограничены потолком.
- ФСП поднимает кандидата внутри категории и не влияет на грейд.
- Кандидат без истории ФСП не штрафуется.

Валидация (синтетика с известной «скрытой истиной», 3000 кандидатов)

Что проверяли	Результат
Грейд совпал с истинным	**77,5%** (самооценка — 63%); ошибка ≤ 1 грейда — 100%
Повторный тест на новых экземплярах	к = **0,85**
Утечка ключа к 40% заданий: доля «нечестных» с повышенным грейдом	статичный тест **54%** → наш тест **0%**
Точность топ-10 подборки (P@10)	поиск по ключевым словам 0,54 → **наш скоринг 0,82**
Детектор утечек	3,7% ложных срабатываний; 65–80% обнаружения

Полный отчёт с методикой и ограничениями — `docs/validation/report.md`.

Соответствие ответам постановщиков (Q&A)

Ответ на сессии	Как реализовано
Грейд — только по тесту категории	Грейд присваивает CAT; ФСП и резюме на него не влияют (02)
ФСП — ранжирование внутри категории, без иерархии соревнований	Сила профиля и скоринг; результат × давность, потолок на участия (04)
Неподтверждённый грейд — показывать и опускать	Отдельный хвост выдачи с пометкой (03)
Кандидат сам выбирает грейд, повышение — отдельным тестом, кулдаун	Политика попыток с обоснованием (02)
Одинаковые задачи не должны расходиться	Параметрические семейства, экспозиция, детектор утечек (02)
Жюри проходит тесты само, экспресс 3–5 минут	Одноразовый демо-кандидат и экспресс-режим (02)
LLM — российские, использование описать, GPU нет	GigaChat / YandexGPT, необязательны, без GPU (07)
FSP ID и Keycloak — имитация	OIDC через Keycloak (realm `fsp`) и мок реестра (04)
Чужие вакансии не импортировать	Только вакансии и потребности работодателей платформы
Контакты открываются после приглашения	Контакты видны только после принятия или отклика (03)
Зарплата: подписать gross или net	Везде «₽/мес до НДФЛ» (03)
Стенд: ~8 CPU, 32–64 ГБ, без GPU	Стек под нагрузкой ~1 ГБ RAM, GPU не нужен (08)

ФСП Талант

Платформа подбора ИТ-специалистов с **обратной механикой**. Кандидат проходит опрос и адаптивный тест и получает категорию (специализация × грейд). Работодатель описывает потребность, получает подборку с объяснением и сам приглашает кандидата: с описанием предложения и обязательной зарплатной вилкой.

Решение для ЛЦТ-2026, специальный трек Федерации спортивного программирования.

Что внутри

- **Тест на грейд, устойчивый к утечкам.** Задания — параметрические *семейства*: у каждого кандидата свой экземпляр, правильный ответ вычисляется исполнением кода, SQL-движком или точной моделью семантики. Модель IRT (3PL) и адаптивный отбор заданий (CAT) с балансом навыков и контролем экспозиции. Детектор утечек на CUSUM.
- **Объяснимый подбор.** Работодатель описывает потребность текстом, NLP разбирает его в структуру. Система рекомендует категории и ранжирует кандидатов; к каждой карточке приложено объяснение вкладов.
- **Честный выход на контакт.**
- Анонимная витрина до согласия кандидата.
- Проверка «вилка попадает в ожидания» без раскрытия суммы.
- Квота приглашений.
- Отказ с причиной, который адаптирует веса ранжирования компании.
- **ФСП ID и достижения.** Привязка через OIDC (Keycloak, realm `fsp`). Мок реестра ФСП реализует предложенный нами контракт данных. Кандидат без истории ФСП ничего не теряет.
- **Плюс к обязательному:**
- отклики на вакансии;
- регулярные задания с оценкой по рубрике;
- PDF-профиль с QR-проверкой подлинности;
- траст-скор работодателя;
- вебхуки для ATS;
- адаптивная вёрстка.
- **Валидация.** Монте-Карло симуляция на популяции с известной «скрытой истиной». Отчёт с результатами: docs/validation/report.md.

Быстрый старт

Нужен Docker с Compose v2 и около 3 ГБ свободной памяти.

```
git clone <repo> fsp-talent && cd fsp-talent
docker compose up -d --build
```

Через 1–2 минуты откройте http://localhost:8080. При первом запуске в базу загружаются демо-данные: 500 кандидатов, 15 компаний, вакансии и история приглашений.

Адрес	Что там
http://localhost:8080	Веб-приложение
http://localhost:8080/api/docs	Swagger (OpenAPI)
http://localhost:8080/auth	ФСП ID (Keycloak), админка: `admin` / `admin`
http://localhost:8080/mail	Тестовый почтовый ящик (письма подтверждения email)

Демо-доступ

Пароль для всех демо-аккаунтов — `demo12345`. На странице входа есть кнопки быстрого входа.

Аккаунт	Роль
`demo.employer@fsp-talent.ru`	Работодатель «Кодовая Гавань»: потребности, подборки, вакансии, задания, история приглашений
`demo.candidate@fsp-talent.ru`	Кандидат Middle Backend с ФСП ID, приглашениями и заданием недели
`demo.new@fsp-talent.ru`	Новый кандидат: пройти опрос и тест с нуля

Тестовые участники в ФСП ID: `fsp-100231`, `fsp-123457`, `fsp-200001`, `fsp-300010` (без истории), пароль `fsp12345`.

Все компании и люди в демо-данных вымышлены.

Сквозной сценарий для проверки (5–7 минут)

1. **Регистрация кандидата.** «Регистрация» → «Я ищу работу» → подтвердите email: ссылка есть в ответе и в `/mail`.
2. **Опрос и тест.** Согласия → специализация → стек → самооценка → грейд, затем адаптивный тест из 10–18 заданий. Результат: категория, интервал оценки, профиль по навыкам, разбор заданий.
3. **ФСП ID.** «Настройки» → «Привязать ФСП ID» → вход в Keycloak (`fsp-123457` / `fsp12345`) → достижения загружаются из реестра.
4. **Подбор у работодателя.** Войдите как работодатель → «Новая потребность» → опишите задачу текстом → «Разобрать текст» → «Найти кандидатов». Дальше:
5. посмотрите рекомендованные категории, объяснения и фильтры;
6. нажмите «Пригласить»: система проверит, попадает ли вилка в ожидания кандидата, после чего можно отправить приглашение.
7. **Ответ кандидата.** Войдите как кандидат → «Приглашения» → «Принять»: у работодателя откроются контакты. Или отклоните с причиной: веса подбора компании адаптируются (страница «Компания»).
8. **Классический отклик.** «Вакансии» → «Откликнуться» → у работодателя отклик появится с контактами.
9. **Механики вживую.** На странице http://localhost:8080/how вставьте текст вакансии: система покажет blueprint теста и сгенерирует задания. Повторная генерация даёт новые экземпляры.

Документация

Раздел ТЗ 3.4	Документ
Функциональная и компонентная архитектура	docs/01-architecture.md
Механика тестирования и устойчивость к утечкам	docs/02-testing.md
Механика подбора и категоризации	docs/03-matching.md
Процедура валидации и результаты	docs/validation/report.md
Схема интеграции с реестром ФСП	docs/04-fsp-integration.md
Описание API	docs/05-api.md и Swagger `/api/docs`
Концепция за рамками MVP: антифрод и ATS	docs/06-antifraud-ats.md
Библиотеки и версии	docs/07-libraries.md
Сборка, развёртывание, локальный запуск	docs/08-deploy.md

Структура репозитория

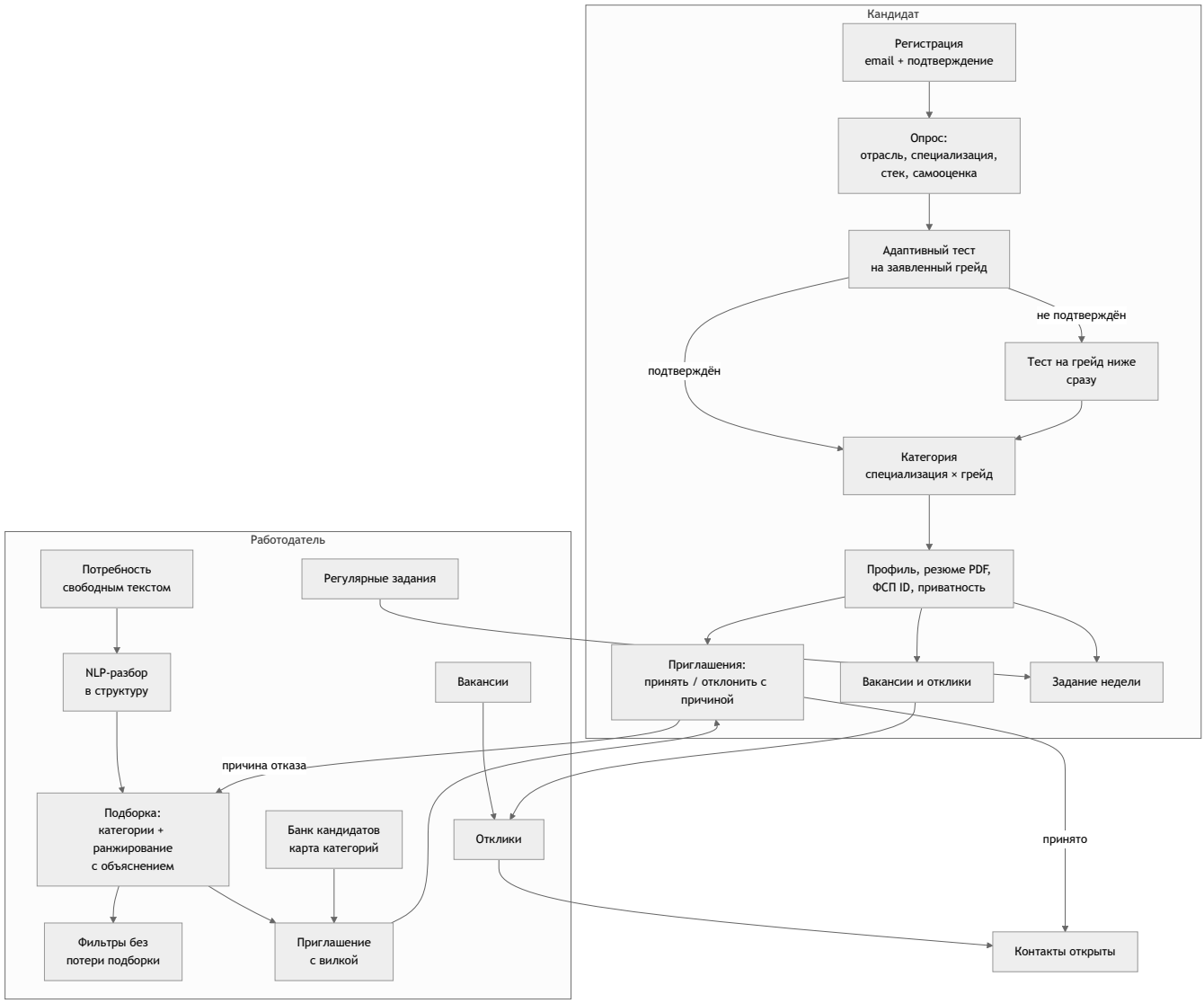
```
backend/            FastAPI, SQLAlchemy, Alembic; модули: assessment (IRT/CAT + семейства
заданий),
                    matching, invitations, fsp, resume, trust, tasks; seed (синтетика),
validation, tests
frontend/           React + TypeScript + Vite + Tailwind; кабинеты кандидата и работодателя
deploy/keycloak/    realm «fsp» — ФСП ID
docs/               документация и отчёт валидации
docker-compose.yml  postgres, backend, fsp_registry, keycloak, mailpit, frontend (nginx)
```

Лицензия

Исходный код — MIT. Шрифты PT Sans, Golos Text и JetBrains Mono распространяются по OFL.
Логотипы ФСП взяты из брендбука, предоставленного организаторами для хакатона.

1. Архитектура решения

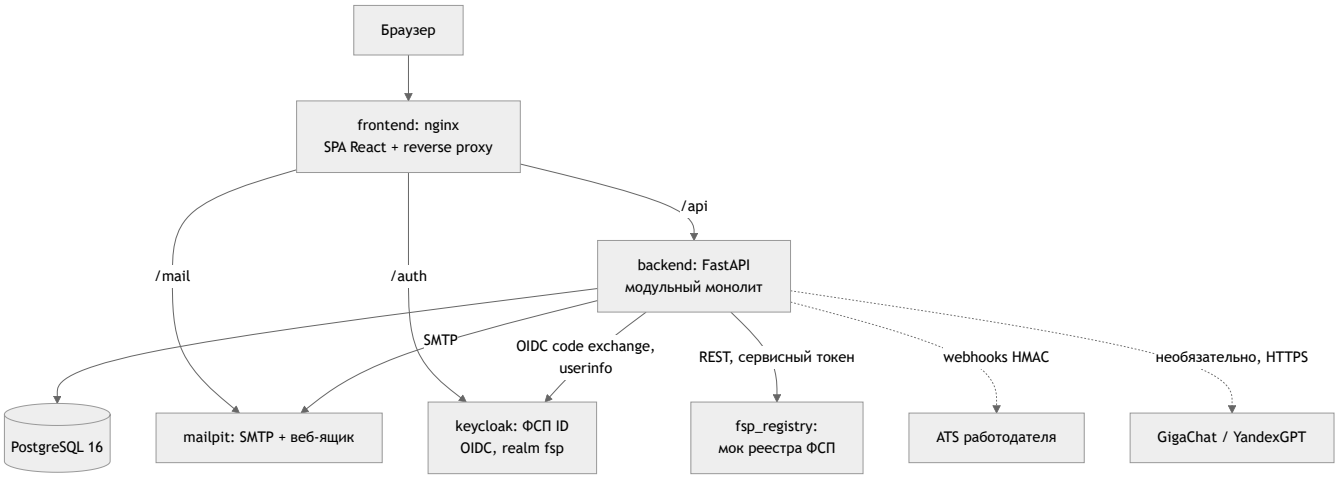
Функциональная архитектура



Роли и права:

Роль	Видит	Не видит
Кандидат	свой профиль, свои приглашения, отклики и задания, опубликованные вакансии	данные других кандидатов
Работодатель	обезличенные карточки кандидатов с грейдом, навыками, ФСП; свои потребности, приглашения, вакансии и отклики	ФИО, контакты, возраст и фото кандидата — до принятия приглашения или отклика; точную сумму ожиданий
Гость	лендинг, механики, публичную проверку PDF-профиля по QR	кабинеты

Компонентная архитектура

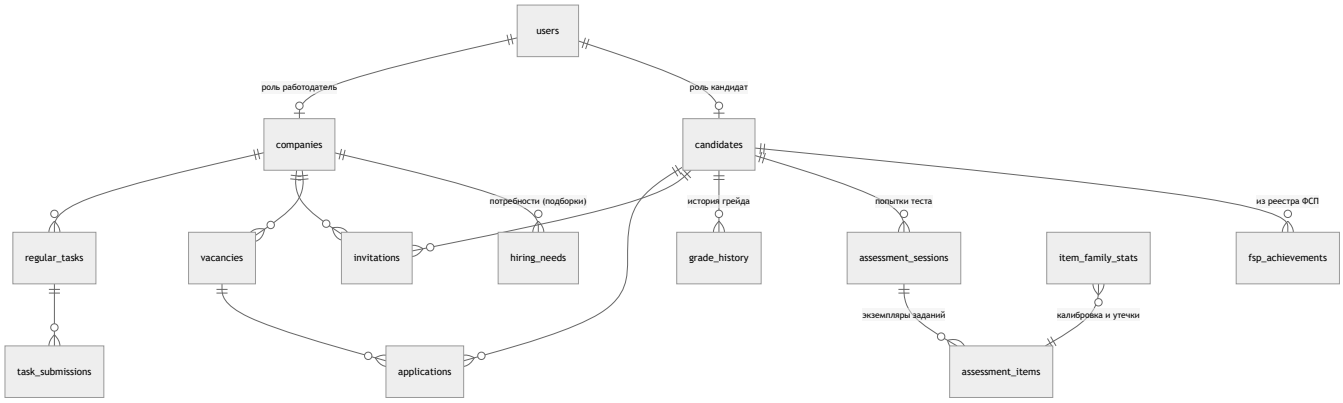


Backend — **модульный монолит** с явными границами модулей. Это осознанный выбор для MVP: один процесс проще развернуть и отладить, а модули слабо связаны и выносятся в сервисы без переписывания. Реестр ФСП уже работает отдельным сервисом: интеграция проверена по сети, а не вызовом функции.

Модуль (`backend/app/modules`)	**Ответственность**
`assessment/families`	59 параметрических семейств заданий (150 позиций «семейство × уровень»), генераторы с вычисляемыми ответами
`assessment/irt.py`, `cat.py`	3PL IRT, EAP-оценка, адаптивный отбор, правило остановки, решение о грейде. Чистые функции без БД
`assessment/service.py`	Политика смены грейда, сессии, приём ответов, детектор утечек (CUSUM), калибровочные счётчики
`matching/scoring.py`	Объяснимый скоринг «кандидат ↔ потребность», сила профиля внутри категории, адаптация весов
`matching/service.py`	Подборки, категории, фильтры, анонимные карточки, правило раскрытия контактов
`nlp.py`	Извлечение навыков, грейда, специализации, вилки, формата и города из текста; символьная близость текстов
`llm.py`	Необязательный слой GigaChat / YandexGPT поверх правил: уточнение разбора с проверкой по справочнику, фолбэк на правила
`invitations.py`	Жизненный цикл приглашений и откликов, квота, причины отказов, вебхуки
`fsp/`	ФСП ID (OIDC), клиент реестра, мок реестра, скоринг достижений
`resume.py`, `pdf_profile.py`	Разбор PDF-резюме; стандартизированный PDF-профиль с QR
`trust.py`	Траст-скор работодателя (защита от фиктивных вакансий)
`tasks.py`	Регулярные задания, оценка по рубрике, индекс актуальности
`events.py`	Уведомления в приложении и исходящие вебхуки

Слои: `api/` (HTTP, схемы Pydantic, права) → `modules/` (бизнес-логика) → `models/` (SQLAlchemy) → PostgreSQL. Скоринг и движок теста — чистые функции: продукт и офлайн-валидация (`backend/validation`) выполняют **один и тот же код**.

Модель данных (основное)



Безопасность и 152-ФЗ

- **Пароли:** argon2id. **Токены:** JWT HS256 с ограниченным сроком. Одноразовые токены подтверждения email хранятся только в виде SHA-256.
- **Разграничение прав по ролям** на уровне зависимостей FastAPI (`current_candidate` / `current_company`). Доступ к чужим объектам возвращает 404, чтобы не раскрывать само существование объекта.
- **Согласия:** два отдельных согласия — на обработку и на публикацию обезличенного профиля, с датами. Без согласия на публикацию кандидат не попадает в выдачу.
- **Право на удаление:** удаление аккаунта каскадно стирает все персональные данные.
- **Минимизация данных:**
- работодатель не получает ФИО и контакты до согласия кандидата;
- точная сумма ожиданий не раскрывается никогда, работодатель видит только «попадает / не попадает»;
- названия прошлых работодателей скрыты по умолчанию.
- **ФСП ID:** подписанный параметр `state` против CSRF, обмен кода на сервере, `client_secret` не покидает backend.
- **Вебхуки:** подпись HMAC-SHA256.

Производительность

Подборка строится в памяти по кандидатам целевой и соседних категорий (сотни записей), формирование занимает десятки миллисекунд. Выбор следующего задания — O(размер банка). Для масштаба в десятки тысяч кандидатов в категории путь развития такой: - предварительный отбор в БД (индексы по `specialization`, `grade`, GIN-индекс по навыкам); - кэш карточек; - вынос скоринга в отдельный воркер.

2. Механика тестирования кандидатов

Задача и требования ТЗ

Нужно получить категорию (специализация × грейд) не из самоописания, а через опрос, выбор предполагаемого грейда и тест. При этом: 1. единый набор заданий не подходит: задания быстро «разойдутся» по рынку; 2. генерация уникальных заданий нейросетью не гарантирует сопоставимой сложности и объективности; 3. грейд не понижается принудительно; смену грейда нужно ограничить по времени; 4. нужны регулярные короткие задания от работодателей.

Решение: параметрические семейства + адаптивный тест (IRT/CAT)

1. Семейство заданий вместо задания

Семейство — это **шаблон** со стабильной структурой и случайной «поверхностью». Генератор получает случайное зерно и подставляет данные, имена, константы и ветви логики. **Правильный ответ вычисляется**: - исполнением сгенерированной программы — Python-семейства; - SQL-движком — SQLite в памяти; - точной моделью семантики: симулятор event loop JS, калькулятор специфичности CSS, модель фаз DOM-событий, формулы HPA, cron и umask.

Ответ никогда не пишется вручную и не придумывается LLM.

Пример: семейство `js_event_loop`, уровень 3. Каждый кандидат получает свою программу из `setTimeout`, промисов и вложенных микрозадач. Ответ — порядок вывода, который вычисляет симулятор очередей.

Свойство	Как достигается
Ответ нельзя «слить»	У кандидата свой экземпляр: медианная вероятность совпадения двух экземпляров — см. отчёт валидации, раздел 5
Сопоставимая сложность	Структура семейства и уровня фиксирована, меняется только поверхность. У уровня свои параметры IRT
Объективность	Ответ вычисляется детерминированно; проверка нечувствительна к пробелам и виду кавычек
LLM не нужна для оценки	LLM (если подключена) можно использовать только для перефразирования условия; ответ проверяет генератор

Банк: **59 семейств, 150 позиций** «семейство × уровень», 23 навыка, 5 специализаций (Backend, Frontend, Data/ML, QA, DevOps). Форматы: предсказать вывод, найти нарушение контракта, оценить сложность алгоритма, посчитать (подсети, мощности, HPA, cron), выбрать архитектурное решение по сценарию.

Банк расширялся по данным валидации. На 92 позициях тест из 18 заданий исчерпывал пул специализации: кандидат QA, DevOps или Data видел почти все позиции своего пула. Добавлен 21 семейство для самых нагруженных навыков (тест-дизайн, ML, статистика, Docker, Kubernetes, CI/CD, TypeScript, React, браузер, HTTP, SQL), а бедные шаблоны расширены так, что вероятность совпадения экземпляров у **каждой** позиции ниже 1% (тест `test_families.py`).

2. Модель IRT 3PL

Вероятность верного ответа на позицию i:

$P(\theta) = c_i + (1 - c_i) / (1 + e^{(-a_i(\theta - b_i))})$

Параметры: - **b_i** — сложность; - **a_i** — дискриминативность; - **c_i** — вероятность угадать: 1/k для выбора из k вариантов, 0 для открытого ответа.

Начальные a и b задаёт эксперт по уровню семейства. По мере накопления ответов они калибруются: в `item_family_stats` копятся экспозиции, ответы и доля верных.

3. Адаптивный отбор (CAT)

- **Оценка способности** — EAP по сетке из 161 точки с априорным N(0,1). Она устойчива на коротких тестах и при ответах «всё верно» / «всё неверно».
- **Следующее задание:**
- выбирается навык, который сильнее всего отстаёт от своей доли в **blueprint**;
- среди его позиций берётся максимум информации Фишера в текущей оценке θ;
- **контроль экспозиции** (онлайн-вариант Sympson–Hetter): позиция, показанная в доле r > 0,4 сессий *своей специализации*, допускается с вероятностью 0,4 / r. Доли берутся из БД (`service.exposure_rates`), первые 50 сессий специализации контроль не действует. Если отсеялось всё — берётся исходный пул, сессия не встаёт;
- итоговая позиция выбирается случайно из топ-K по информации (randomesque), K сужается с 5 до 2 по мере уточнения оценки;
- не больше двух уровней одного семейства за попытку.

Порог 0,4 подобран перебором (отчёт валидации, раздел 3): максимальная экспозиция позиции внутри специализации падает с ~94% до ~81% без потери точности. Ниже не опустить: квота навыка в blueprint задаёт минимум. Поэтому главная метрика — не экспозиция позиции, а произведение «экспозиция × вероятность совпадения экземпляра». - **Blueprint** — доли навыков в тесте: - по умолчанию 70% — ядро специализации, 30% — заявленный стек кандидата; - для теста по вакансии 70% — навыки вакансии, 30% — ядро.

Навык без заданий отображается на ближайший навык по графу сходства (например, `django` → `python`). - **Остановка теста.** Тест заканчивается при любом из условий: - SD апостериорного распределения ≤ 0,33; - решение о грейде уже однозначно: P(≥ порога) ≤ 3% либо ≥ 97% при P(≥ следующего) ≤ 10%; - набрано 18 заданий.

Минимум — 10 заданий. Обычно тест занимает 10–18 заданий. - **Время:** у каждой позиции свой лимит (60–240 с) плюс 15 с допуска на сеть. Просроченный ответ засчитывается как неверный.

4. От θ к грейду

Пороги на шкале θ (популяция N(0,1)): Junior ≥ −0,9, Middle ≥ 0,1, Senior ≥ 1,1, ниже — Intern.

Итог попытки	Условие	Что происходит
`confirmed`	P(θ ≥ порог заявленного) ≥ 0,6	Категория присвоена
`upgrade_available`	И P(θ ≥ порог следующего) ≥ 0,6	Категория присвоена, тест на грейд выше открыт 7 дней
`not_confirmed`	Иначе	Грейд **не понижается**: прежний сохраняется, сразу доступен тест на грейд ниже

Порог 0,6 выбран на симуляции: он балансирует точность и долю завышений, подробности в отчёте валидации.

Заявленный грейд не влияет на оценку. Он задаёт только стартовую точку отбора заданий. Априорное распределение одинаковое для всех, поэтому «заявить Senior на удачу» бесполезно. Это проверено в валидации.

5. Политика частоты

Правило	Значение
Тест на грейд ниже после неудачи	Сразу
Повтор того же грейда после неудачи	Через 30 дней
Подтверждение текущего грейда (обновить результат)	Раз в 30 дней
Смена подтверждённого грейда (вверх или вниз)	Не чаще раза в 90 дней
Исключение	После `upgrade_available` тест на грейд выше доступен 7 дней

Ограничения проверяет сервер (`GET /api/assessment/eligibility`). Интерфейс показывает причину и дату, с которой попытка станет доступна.

Почему так: - **Провал не сжигает попытку.** Тест на ступень ниже доступен сразу: кандидат не должен выпадать из оборота из-за завышенной самооценки. Прежний подтверждённый грейд при провале сохраняется. - **Повышение — только отдельным тестом.** Автоматического прыжка Junior → Middle нет: уверенный результат открывает тест на ступень выше на 7 дней, и грейд меняется, только если этот тест его подтвердит. - **Нельзя скакать по грейдам.** Подтверждённый грейд меняется не чаще раза в 90 дней, повтор того же грейда — через 30 дней. Иначе грейд превращается в лотерею, а работодатель видит сегодня Senior, завтра Middle. - **Повторная попытка — всегда новые экземпляры заданий.** Семейства параметрические, поэтому запоминание ответов прошлой попытки не помогает.

Пока ни один грейд не подтверждён, кандидат остаётся в выдаче работодателя с пометкой «заявлен X · не подтверждён», ниже всех подтверждённых кандидатов (см. механику подбора).

Экспресс-режим для проверки жюри

Вход «Новый кандидат» на странице входа (`POST /api/auth/demo-login?role=candidate_new`) каждый раз создаёт новый одноразовый аккаунт. Каждый эксперт проходит опрос и тест сам и не упирается в ограничения частоты чужих попыток. Для демо-аккаунтов тест короче: 6–10 заданий вместо 10–18, это 3–5 минут. Алгоритм тот же, но останов наступает раньше, поэтому погрешность θ выше. Точность в отчёте валидации измерена на обычном режиме.

6. Защита целостности

Защита многоуровневая: 1. **Параметризация:** слитый ответ бесполезен. 2. **Большой пул + контроль экспозиции + лимит семейства:** у кандидатов разные наборы позиций. 3. **Детектор утечек.** По каждой позиции считается CUSUM остатков «наблюдаемая − ожидаемая» доля верных ответов при оценённой способности отвечающих: $S \leftarrow \max(0, S + (x_i - P(\hat{\theta}_i)) - 0{,}15)$. При $S > 3$ и не менее 20 ответах позиция получает статус `flagged` и выходит из ротации до проверки. Если семейство «раскусили» (появился разбор, как решать), оно уходит из ротации, а автор добавляет новую поверхность или уровень. 4. **Сигналы:** время ответа, вставка из буфера, уход со вкладки, копирование. Они фиксируются в отчёте сессии (`flags`), но **не штрафуют автоматически**: ложные срабатывания недопустимы. 5. **До конца теста** кандидат не видит правильность ответов: ответ на одно задание не подсказывает следующие. 6. **Жалоба на задание.** В разборе после теста у каждого задания есть кнопка «Задание некорректно» (причина и комментарий в `AssessmentItem.signals.report`). Жалобы попадают в выгрузку пилота и в отчёт валидации.

7. Пилот на людях

После теста кандидат может заполнить необязательную анкету: стаж, должность, самооценка, грейд по оценке тимлида (`POST /api/assessment/sessions/{id}/survey`). Обезличенная выгрузка (`GET /api/assessment/pilot/export`, заголовок `X-Pilot-Token = PILOT_EXPORT_TOKEN`) обрабатывается `validation/pilot.py`: согласие $\hat{\theta}$ со стажем и оценкой тимлида (ρ Спирмена, каппа), калибровка позиций (наблюдаемая и ожидаемая доля верных), время, жалобы.

8. Тест по вакансии

`POST /api/assessment/blueprint-preview` и `POST /api/assessment/sessions` с `vacancy_id`: навыки вакансии (обязательные с весом 1, «будет плюсом» с весом 0,4) формируют blueprint. На странице «Как это работает» можно вставить текст вакансии и увидеть, как меняется состав теста и какие экземпляры получит кандидат. Повторная генерация даёт новые экземпляры той же сложности.

9. Регулярные задания

Раз в неделю кандидат с подтверждённым грейдом получает короткую задачу от работодателя своей специализации: решить или предложить подход. Задачи подбираются с учётом стека кандидата. Ответ оценивается по прозрачной рубрике: ключевые пункты с маркерами, кандидат видит, какие пункты он раскрыл. Задача может прийти и как «челлендж» внутри приглашения.

Маркеры не ловят ответ, написанный своими словами. Если подключена языковая модель, она решает по каждому пункту рубрики, раскрыт ли он по смыслу, и обязана привести цитату из ответа. Пункт засчитывается, только если цитата действительно есть в тексте ответа. Итоговый балл считает код по весам рубрики, а не модель.

Результат обновляет индекс актуальности профиля: он входит в ранжирование и виден работодателю («подтверждён активностью N дней назад»).

Опрос

Шаги: 1. Согласия. 2. Специализация и отрасли. 3. Стек с годами, общий опыт, формат, роли. 4. Три вопроса самооценки: самостоятельность, масштаб решений, роль в код-ревью. 5. Выбор предполагаемого грейда.

Система подсказывает грейд по ответам и опыту, но решение за кандидатом. Ответы опроса сохраняются в истории и **не влияют на категорию**: её определяет только тест.

3. Механика подбора и категоризации

Категоризация

Категория — это пара **специализация × грейд**, например «Backend · Middle». Её определяют опрос (специализация) и адаптивный тест (грейд). Справочник: - **специализации:** Backend, Frontend, Data/ML, QA, DevOps; - **грейды:** Intern, Junior, Middle, Senior; - **навыки:** 60 позиций с синонимами и графом сходства, `backend/app/catalog.py`.

Выдача работодателю всегда строится по категориям, присвоенным тестом. Самоописание (стек, опыт) уточняет ранжирование, но не определяет категорию.

Внутри категории кандидаты ранжируются по **силе подтверждённого профиля**:

strength = 0,60 · положение θ внутри полосы грейда + 0,25 · сигнал ФСП + 0,15 · актуальность.

Достижения ФСП двигают кандидата **только внутри категории** и на грейд не влияют: грейд даёт только тест.

Неподтверждённый грейд

Кандидат, который проходил тест, но ещё не подтвердил ни одного грейда, не скрывается: иначе он выпал бы из оборота. Такие кандидаты: - попадают в категорию заявленного грейда (из последней неуспешной попытки); - идут **после всех подтверждённых** в подборке, поиске и категории, под разделителем «Грейд не подтверждён тестом»; - помечены бейджем «заявлен X · не подтверждён» и чипом объяснения с минусом; - получают компонент «Грейд» = 0 и силу профиля ×0,5.

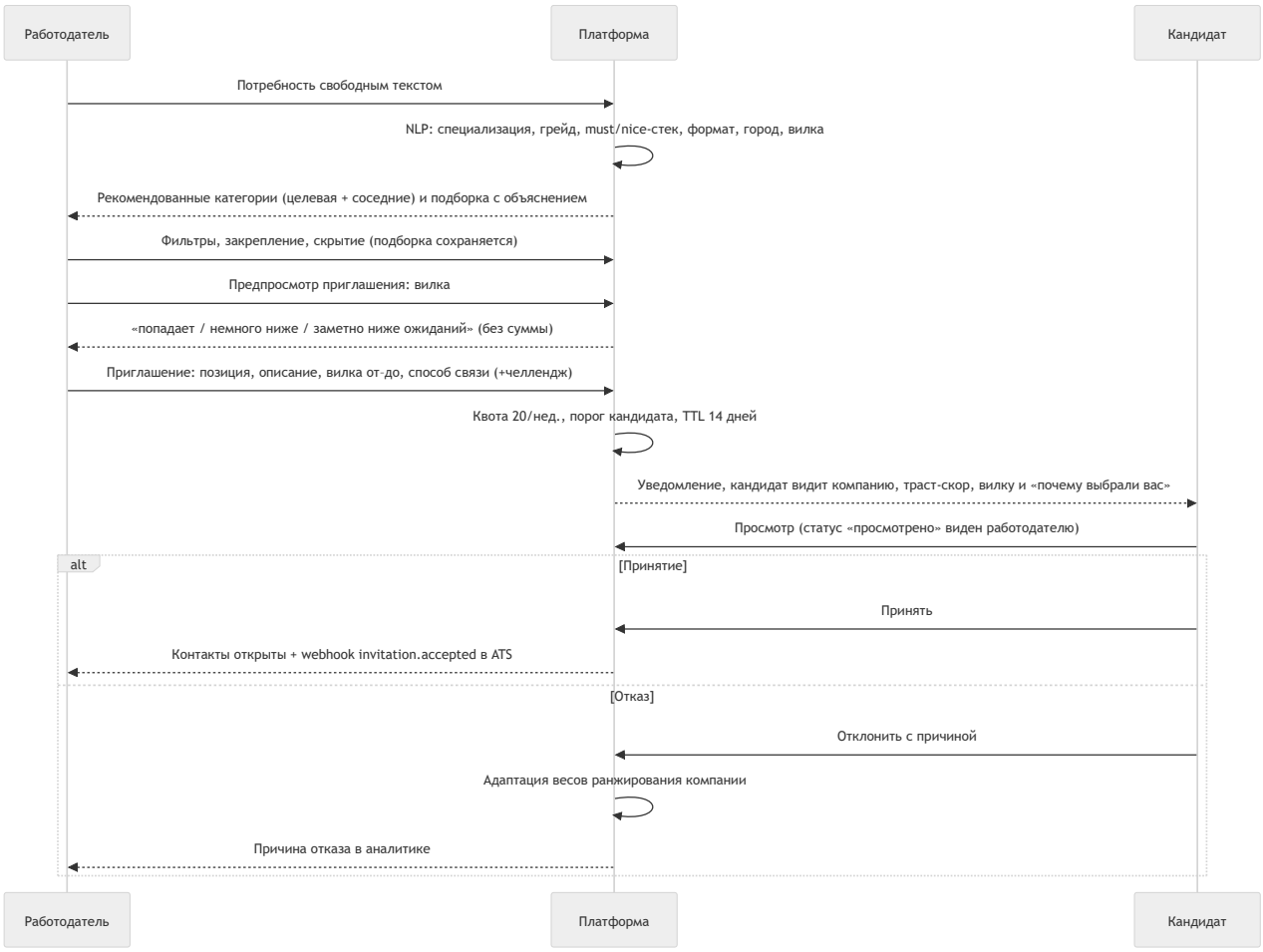
Пригласить такого кандидата можно, но предпросмотр приглашения предупреждает, что грейд не подтверждён. После успешной пересдачи пометка снимается.

Зарплаты

Все суммы на платформе — в месяц **до вычета НДФЛ (gross)**: вилка в вакансии и приглашении, ожидания и порог кандидата. Так сравнение вилки с ожиданиями корректно, а подписи в интерфейсе это явно указывают.

Сценарий выхода на контакт

Ориентир — модель LHH: инициатива у работодателя. Мы дополнили её механизмами, которые делают контакт честным для обеих сторон.



Чем механика отличается от прямого отклика и витрины резюме:

Механизм	Зачем
Анонимная витрина	Работодатель видит подтверждённые навыки, грейд, ФСП и опыт без названий компаний. ФИО, возраст, фото и контакты скрыты. Меньше дискриминации, соответствие 152-ФЗ
Вилка обязательна и видна до разговора	Кандидат оценивает предложение до раскрытия контактов (ТЗ, п. 2.2.1)
Проверка вилки без раскрытия суммы	Работодатель заранее знает, попадает ли он в ожидания, и не тратит приглашение впустую
Порог кандидата	Приглашения ниже минимальной суммы автоматически помечаются; работодатель эту сумму не видит
Квота приглашений	20 в неделю на компанию, персональный текст обязателен. Против массовых рассылок
Отказ с причиной → адаптация весов	Причина («зарплата», «стек», «формат», «грейд») увеличивает вес соответствующего компонента в будущих подборках этой компании (×1,1 за отказ, не больше ×2)
Приглашение-челлендж	К приглашению можно приложить короткое задание; решение работодатель увидит в разделе «Задания»
Траст-скор работодателя	Кандидат видит уровень доверия к компании в каждом приглашении (см. концепцию антифрода)
Классический отклик	Кандидат без приглашений может откликнуться сам; отклик тоже открывает контакты

Статусы приглашений: `sent → viewed → accepted | declined`; кроме того, `expired` по истечении TTL и `withdrawn` при отзыве работодателем. Статусы откликов: `sent → viewed → invited | rejected`, а также `withdrawn`. Обе стороны видят актуальный статус своих записей.

Ранжирование по потребности

`backend/app/modules/matching/scoring.py`. Это сумма вкладов, каждый компонент лежит в [0, 1]:

Компонент	Вес	Как считается
Стек	0,32	0,8 × покрытие обязательных + 0,2 × желательных. Навык, подтверждённый тестом на нужном уровне, = 1,0; заявленный = 0,6 (до 0,9, если подтверждён близкий навык); отсутствующий навык получает частичный зачёт за близкий по графу (Django ≈ FastAPI ≈ Flask)
Грейд	0,20	Совпадает = 1; выше на ступень = 0,7; ниже на ступень = 0,45
Результат теста	0,16	Сигмоида от (θ на нужных навыках − порог запрошенного грейда)
Достижения ФСП	0,05	Насыщающаяся сумма: результат × давность по каждому соревнованию (без иерархии уровней и дисциплин, участия без места ограничены потолком) + разряд
Зарплата	0,11	Ожидания ≤ верхней границы = 1; до +15% = 0,5; выше = 0,1; не указаны = 0,7
Формат и город	0,08	Пересечение форматов; для офиса и гибрида — город или готовность к релокации
Актуальность	0,05	Полураспад 90 дней от последнего теста или задания
Описание	0,03	Косинус символьных 4-грамм текста потребности и профиля

Объяснимость. Каждая карточка несёт «чипы» со знаком вклада, например: - «+ Подтверждено тестом: Python, Асинхронность»; - «− Нет обязательных: Kafka»; - «FastAPI нет, но есть близкий Django».

Детальная разбивка по компонентам с весами открывается в профиле кандидата. Веса задаются в одном конфиге и подобраны по результатам валидации (раздел 6 отчёта).

Фильтрация без потери подборки. Потребность хранится на сервере. Фильтры (грейд, стек в профиле или подтверждённый тестом, ФСП, формат, город, опыт, потолок ожиданий) сужают ту же выдачу. Закреплённые кандидаты остаются при любых фильтрах, скрытые исключаются. Фильтры сохраняются в сессии браузера для каждой подборки.

Разбор потребности (NLP)

Разбор двухступенчатый. Первая ступень — правила `backend/app/modules/nlp.py`: детерминированно, без внешних зависимостей, работает всегда. Вторая ступень необязательна: если подключена языковая модель (GigaChat или YandexGPT, `backend/app/modules/llm.py`), она уточняет результат правил. Модели не доверяем вслепую: - навыки, специализация, грейд и формат принимаются только как id из справочника, остальное отбрасывается; - точные поля, которые правила извлекают надёжно (вилка, явный грейд, формат, город), остаются из правил; - модель добавляет то, что правила не видят: навыки, описанные словами («знание реляционных баз» → SQL), и разделение «обязательно / желательно» в сложных фразах; - при ошибке сети, таймауте или некорректном ответе остаётся результат правил.

Работодатель видит, каким путём получен разбор («правила» или «правила + языковая модель»), и правит любые поля.

Что извлекается: - **Навыки:** словарь с синонимами и границами слов (`postgres` → PostgreSQL, `k8s` → Kubernetes, `drf` → Django). - **Желательные навыки:** маркеры «будет плюсом / желательно / приветствуется» до или после перечня. Навык, упомянутый и вне такого блока, остаётся обязательным. - **Навыки:** справочник с синонимами и словарь жаргона и транслитерации («постгрес», «кубер», «джанго»); русские слова совпадают в любой форме («на пайтоне», «джангу»); из пересекающихся совпадений берётся самое длинное («GitLab CI» — это CI/CD, а не ещё и Git). - **Грейд:** по ключевым словам (джун, мидл, сеньор, стажёр, ведущий) или по требуемому стажу в любой форме («опыт коммерческой разработки от 3 лет», «опыт от двух лет», «4+ years»). - **Специализация:** по ключевым словам и вкладу навыков в ядро специализации. - **Вилка:** «от … до», «200–300 тыс.», «350-450k», «250к-300к», «до 70 тыс.», «от 200к», одна сумма («з/п 100 000 руб.», «400к») и голый диапазон в конце текста («удалённо 200-260»). - **Формат и город.**

Качество разбора измерено в разделе 7 отчёта валидации на трёх наборах, включая отложенный: 20 текстов, написанных после заморозки правил. Локальные эмбеддинги проверены и не включены: на жаргоне они уверенно ошибаются (подробности и воспроизведение — там же).

Разбор резюме PDF (`resume.py`): PyMuPDF извлекает текст, затем правила находят: - ФИО — 2–3 слова с заглавной буквы, сначала в начале страницы, затем во всём тексте (шаблоны с боковой колонкой); - email, телефон (приводится к +7XXXXXXXXXX), Telegram («@ник», «t.me/ник», «Telegram: ник»); - стаж — объединение интервалов без двойного счёта; форматы «Январь 2020 — настоящее время», «03.2019 – 05.2021», «Sep 2021 – Present», «с 2016 по 2019»; - должности, стек, роли и soft skills.

С подключённой моделью она дополняет ФИО, должности и навыки, описанные в тексте словами. Контакты остаются из правил, навыки и soft skills проходят проверку по справочнику. Кандидат отмечает, какие поля применить к профилю.

Точность по полям — раздел 10 отчёта валидации: 15 резюме в пяти вёрстках и отложенный набор из 6 резюме.

Валидация решения

Отчёт сгенерирован командой `uv run python -m validation.run` (2026-10-07 07:59 UTC). Код валидации — `backend/validation/run.py`; он вызывает **те же модули**, что работают в продукте (`cat.py`, `scoring.py`, `nlp.py`), на синтетической популяции с известной «скрытой истиной» (`app/seed/synth.py`).

Почему синтетика и как она устроена

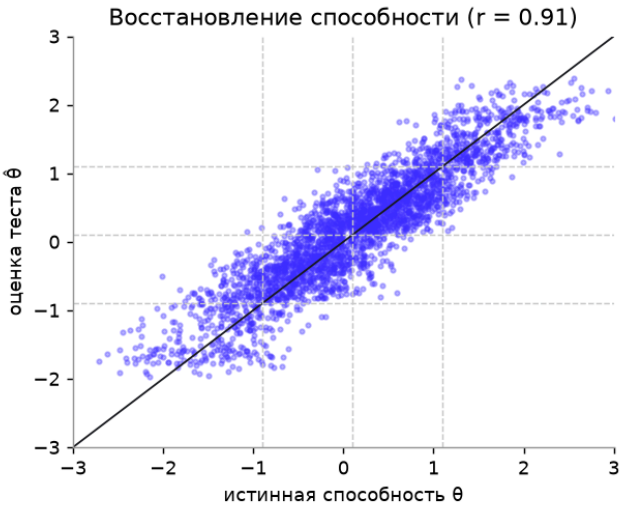
Эталонной разметки на хакатоне нет, поэтому мы моделируем кандидатов, у которых **истинная способность известна**: - у каждого кандидата есть истинная способность θ ~ N(0,15; 0,95) и способности по навыкам θ_s = θ + N(0; 0,45); - наблюдаемое — заявленный грейд (шумная и слегка завышенная самооценка), стек, опыт, ожидания по зарплате — порождается из θ с шумом; - ответы на задания генерируются по модели 3PL с **«истинными» параметрами заданий, отличающимися от экспертных** (b ± 0,25, a × e^N(0; 0,15)), — движок теста их не знает, как и в реальности до калибровки.

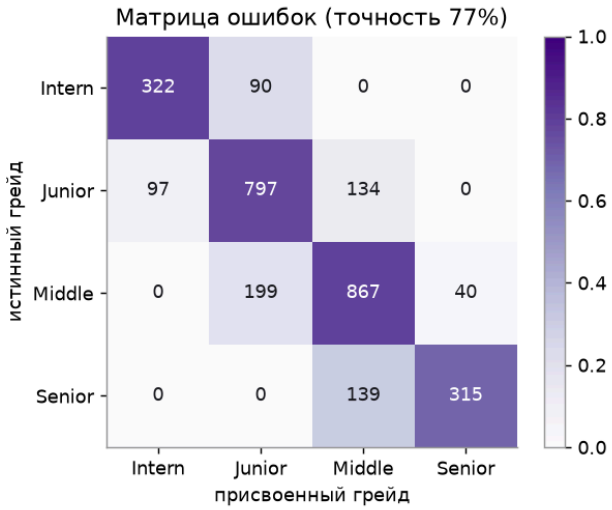
Так мы проверяем то, что важно жюри: восстанавливает ли система скрытую правду, а не совпадает ли сама с собой.

1. Точность присвоения грейда

3000 кандидатов прошли полный сценарий (тест на заявленный грейд → при неуспехе сразу тест на грейд ниже).

Метрика	Значение
Грейд совпал с истинным	**76.7%**
Ошибка не больше одного грейда	**100.0%**
Завышение / занижение	8.8% / 14.5%
Для сравнения: самооценка кандидата совпадает с истиной	63.4% (завышают 24.0%)
Корреляция оценки θ̂ с истинной θ	0.908 (RMSE 0.398)
Заданий за попытку (среднее)	16.7
Попыток до присвоения (среднее)	1.61





Нейтральность к заявленному грейду. Тех же кандидатов прогнали дважды: с честным заявленным грейдом и с завышенным на ступень. Средний сдвиг оценки $\hat{\theta}$ — **-0.049 ± 0.05** (95% ДИ), то есть статистически неотличим от нуля; разброс разницы (SD 0.515) такой же, как между двумя честными попытками (SD 0.446). Заявленный грейд влияет только на выбор стартовых заданий, а оценка строится с общим априорным распределением N(0,1): «заявить Senior на удачу» бесполезно.

Точность решения «подтвердить / не подтвердить». Первая попытка подтверждает заявленный грейд тогда и только тогда, когда истинная способность не ниже порога, в **83.0%** случаев.

Устойчивость к выборке. Те же метрики на 5 независимых популяциях по 1500 человек (сиды 11, 12, 13, 14, 15):

Метрика	Среднее ± SD	По популяциям
Грейд совпал с истинным	77.9% ± 0.6 п. п.	76.9%, 77.8%, 78.5%, 77.9%, 78.4%
Ошибка не больше одного грейда	100.0% ± 0.0 п. п.	100.0%, 100.0%, 100.0%, 100.0%, 100.0%
Взвешенная каппа test-retest	0.842 ± 0.01	0.839, 0.849, 0.849, 0.827, 0.848

Как выбран порог подтверждения. Порог P(θ ≥ порога грейда) перебирается на отдельной популяции (1200 человек, сид 13):

Порог	Грейд совпал	Завышение	Занижение
0.5	76.9%	12.0%	11.1%
0.6	78.2%	7.7%	14.1%
0.7	75.7%	5.6%	18.7%

Выбрано 0.6: работодателю важнее, чтобы подтверждённый грейд не был завышен, а более высокий порог заметно увеличивает занижения. Альтернатива «повторная попытка продолжает измерение» (ответы первой попытки учитываются) экономит около 20% заданий, но снижает точность, поэтому попытки независимы (`synth.CARRY_OVER`).

2. Согласованность (test-retest)

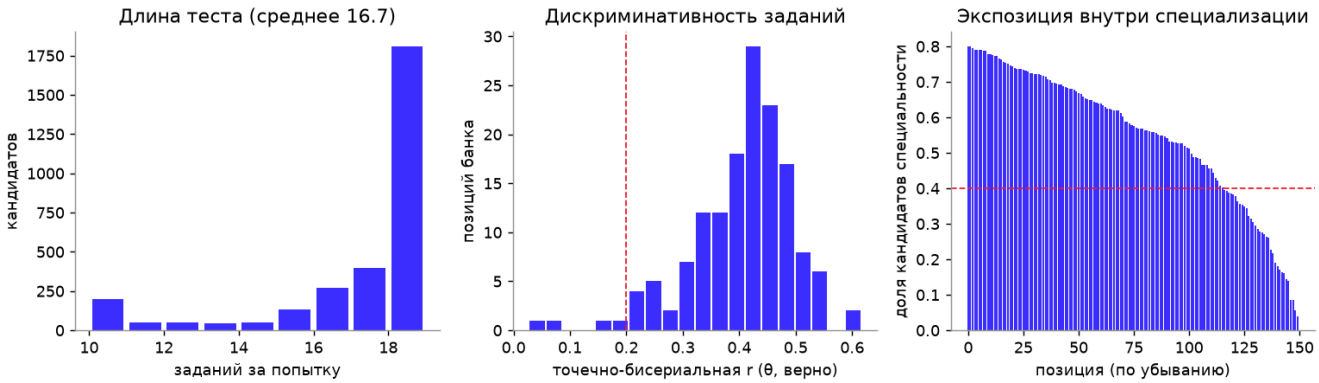
Каждый кандидат прошёл тест повторно с новыми случайными экземплярами заданий.

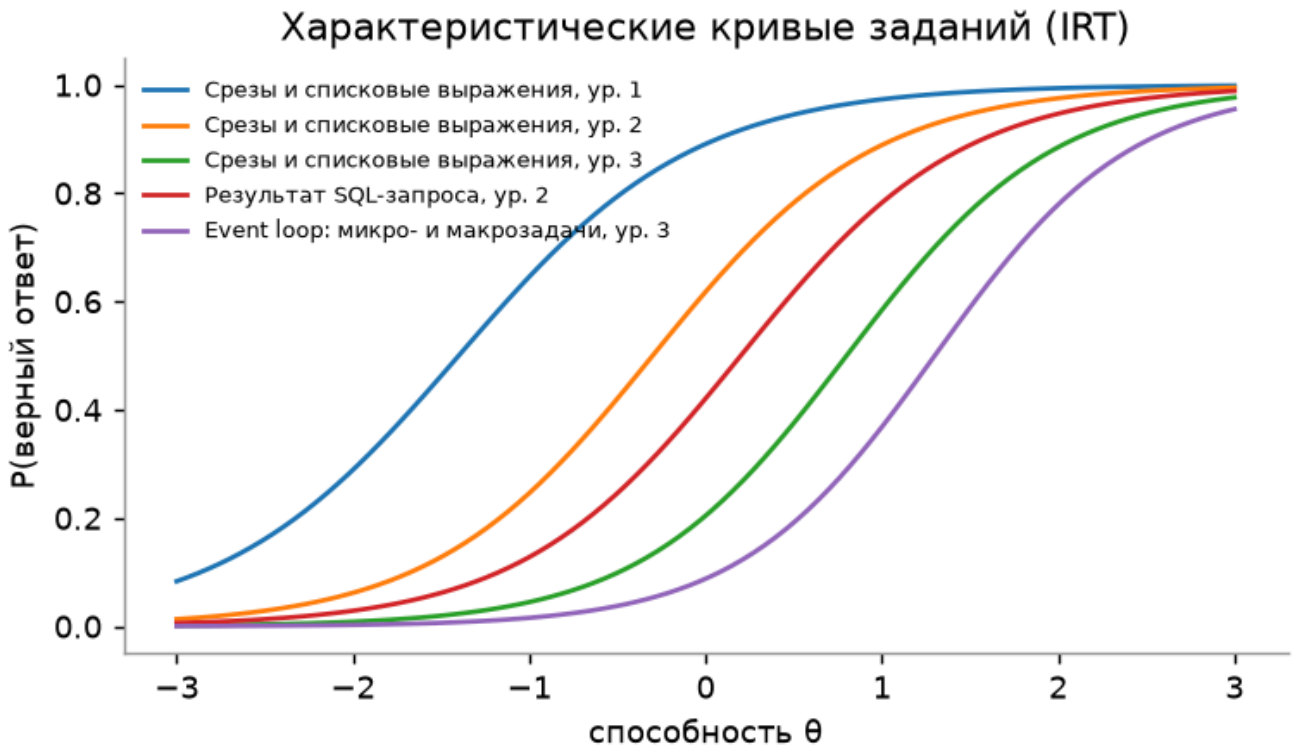
Метрика	Значение
Тот же грейд при повторе	**77.8%**
Взвешенная каппа Коэна	**0.854**
Корреляция $\hat{\theta}$ между попытками	0.874

Квадратично-взвешенная каппа выше 0,8 считается «почти полным согласием».

3. Дискриминативность заданий и экспозиция

Метрика	Значение
Медианная точечно-бисериальная корреляция «способность — верный ответ»	**0.423**
Доля позиций банка с r ≥ 0,2 (порог пригодности задания)	97.3%
Использовано позиций банка	150 из 150





Экспозиция. Утечка распространяется среди кандидатов одной специализации, поэтому главная метрика — доля кандидатов специализации, увидевших позицию (по всем их попыткам). Движок ограничивает её онлайн-вариантом Sympson–Hetter: позиция, показанная в доле r > 0.4 сессий своей специализации, допускается с вероятностью 0.4/r, а randomesque сужается с топ-5 до топ-2 по мере уточнения оценки.

Метрика	Значение
Максимальная экспозиция позиции среди всех кандидатов	43.5%
Максимальная экспозиция внутри специализации	80.0%
Максимум «экспозиция позиции × P(совпадения экземпляра)»	0.68%

Перебор порога r_max (1200 человек, сид 13):

r_max	Грейд совпал	Заданий за попытку	Max экспозиция (все)	Max экспозиция (в специализации)
выключено	78.0%	16.5	47.2%	94.4%
0.6	79.2%	16.6	47.3%	84.9%
0.5	77.7%	16.6	44.3%	83.2%
0.4	78.2%	16.6	44.0%	80.7%
0.3	77.2%	16.9	40.6%	83.2%

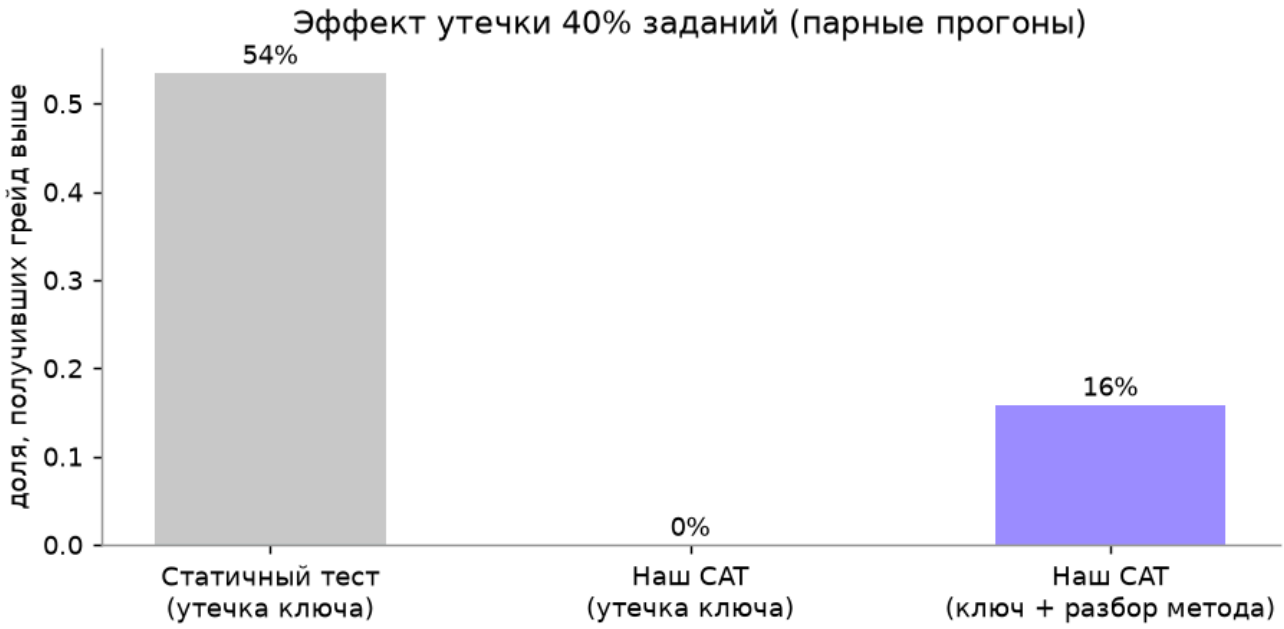
Ниже 80.7% внутри специализации экспозицию не опустить без потери содержательной валидности: квота навыка задаёт минимум. Например, QA-тесту нужно около 6 заданий по тест-дизайну из 10 позиций этого навыка. Поэтому экспозиция *позиции* — не экспозиция *содержания*: каждая сессия получает свой экземпляр семейства, и вероятность, что кандидат увидит тот же экземпляр, что и его знакомый, ограничена произведением в последней строке таблицы выше. Банк расширен до 150 позиций, в том числе чтобы у самых нагруженных навыков (тест-дизайн, ML, статистика, Docker, JavaScript) появился выбор.

4. Устойчивость к утечке заданий

Моделируем утечку 40% заданий. Дизайн **парный**: каждый кандидат проходит тест дважды на одних и тех же случайных числах — честно и со «слитыми» знаниями. Поэтому разница показывает эффект утечки, а не шум повторного теста.

Сценарий	Средний прирост $\hat{\theta}$	Доля получивших грейд выше
Статичный тест (одна форма), утёк ключ ответов	+0.53	**53.5%**
Наш CAT, утёк ключ ответов	+0.00	**0.0%**
Наш CAT, утёк ключ и разбор метода (+0,15 к вероятности на «слитых» семействах)	+0.18	15.9%

В параметрическом тесте слитый ключ бесполезен: новый кандидат получает другие данные и другой правильный ответ. Остаётся «утечка метода» — кандидат заранее разобрался, как решаются такие задачи. Её эффект ограничен, и по сути это освоение навыка.



Детектор утечек (`assessment/leak.py`) — односторонний CUSUM остатков «наблюдаемая − ожидаемая доля верных» относительно собственной базовой линии позиции. Базовая линия нужна, потому что экспертные параметры заданий неточны: без неё любая неоткалиброванная позиция выглядела бы «слитой». Первая версия детектора без базовой линии давала 47% ложных срабатываний; эта проверка её и выявила.

Метрика	Значение
Ложные срабатывания на честном потоке (600 ответов)	5.3%
Обнаружение, ответ знают 30% отвечающих (за 400 ответов)	59.3%, медианная задержка 35 ответов
Обнаружение, ответ знают 50% отвечающих	80.3%, медианная задержка 21 ответов

Детектор — вторая линия защиты. Первая — параметризация, при которой ключ ответов вообще не переносится.

5. Уникальность экземпляров

Для каждой позиции банка (150) сгенерировано 400 экземпляров; вероятность, что два кандидата получат **идентичный** экземпляр, — Σp^2.

Метрика	Значение
Медианная вероятность совпадения	**0.0025**
Доля позиций с вероятностью совпадения < 1%	100.0%
Медианная доля самого частого правильного ответа	17.9%

Наименее вариативные позиции:

Позиция	Различных экземпляров из 400	P(совпадения)
`js_closures:2`	124	0.010
`cron_schedule:1`	136	0.009
`py_classes:3`	153	0.009
`py_iterators:2`	161	0.009
`dom_events:2`	209	0.009

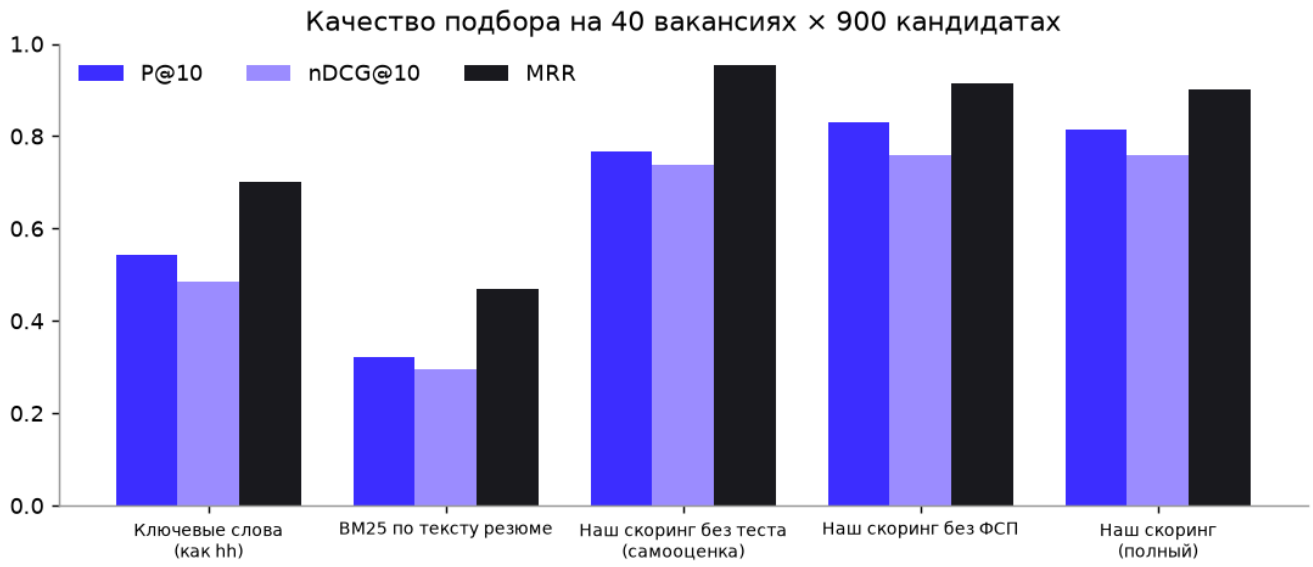
6. Качество подбора

40 вакансий × 900 кандидатов. Релевантность пары (0–3) вычисляется из **скрытой истины**: истинный грейд, реальное владение обязательными навыками на уровне вакансии (заявлено в профиле и истинная θ_s выше порога), совместимость по зарплате и формату. Ни одна система не видит эти величины напрямую. Результаты соревнований ФСП моделируются как шумное измерение способности (результат = θ + N(0; 0,8) → победитель / призёр / финалист / участник, разряд аналогично), участие в ФСП слабо коррелирует со способностью. Медиана «хороших» (rel ≥ 2) кандидатов на вакансию — 41. Все системы ранжируют кандидатов одной специализации.

Система	P@10 (95% ДИ)	nDCG@10 (95% ДИ)	MRR
Ключевые слова (как hh)	0.543 (0.463–0.622)	0.486 (0.433–0.540)	0.700
BM25 по тексту резюме	0.322 (0.253–0.398)	0.296 (0.245–0.350)	0.468
Наш скоринг без теста (самооценка)	0.768 (0.693–0.835)	0.738 (0.690–0.784)	0.954
Наш скоринг без ФСП	0.830 (0.753–0.898)	0.759 (0.711–0.806)	0.915
Наш скоринг (полный)	0.815 (0.733–0.887)	0.760 (0.708–0.809)	0.901

ДИ — перцентильный бутстрап по вакансиям (2000 повторов). Парные разницы считаются по одним и тем же вакансиям, поэтому общая «сложность» вакансии сокращается:

Сравнение	Δ P@10	Δ nDCG@10
Полный − Наш скоринг без ФСП	-0.015 (95% ДИ -0.037…+0.005, в пределах шума)	+0.001 (95% ДИ -0.014…+0.015, в пределах шума)
Полный − Ключевые слова (как hh)	+0.272 (95% ДИ +0.200…+0.348, значимо)	+0.274 (95% ДИ +0.202…+0.345, значимо)
Полный − Наш скоринг без теста (самооценка)	+0.048 (95% ДИ -0.000…+0.092, в пределах шума)	+0.022 (95% ДИ -0.017…+0.060, в пределах шума)



Ключевое: самоописанные данные (ключевые слова, самооценка) ранжируют хуже, чем подтверждённый тестом профиль. Абляция «без теста» показывает вклад механики тестирования в подбор.

Роль ФСП. Для вопроса «подходит ли кандидат вакансии» бонус ФСП почти ничего не добавляет к тесту: тест уже измеряет ту же способность, а кандидат чуть выше порога так же «подходит», как сильный. Разница «полный − без ФСП» выше лежит в пределах шума — в обе стороны. Перебор веса ФСП в оценке соответствия:

Вес ФСП	P@10	nDCG@10
0.0	0.830	0.759
0.03	0.818	0.764
0.05	0.815	0.760
0.1	0.782	0.744

Вес оставлен небольшим (0.05): он не ухудшает подбор и даёт работодателю видимый сигнал. ТЗ отводит ФСП другую роль — **ранжирование внутри категории** по силе подтверждённого профиля. Здесь ФСП — независимое второе измерение, и оно помогает. Корреляция Спирмена порядка внутри категории с истинной способностью (20 категорий, ДИ — бутстрап по категориям):

Порядок внутри категории	ρ Спирмена с истинной θ (95% ДИ)
Сила профиля: тест + ФСП + актуальность	**0.561** (0.521–0.600)
Без ФСП (тест + актуальность)	0.54 (0.499–0.580)
Только ФСП	0.189 (0.123–0.259)

Парная разница «с ФСП − без ФСП» внутри категории: +0.021 (95% ДИ +0.003…+0.044, значимо).

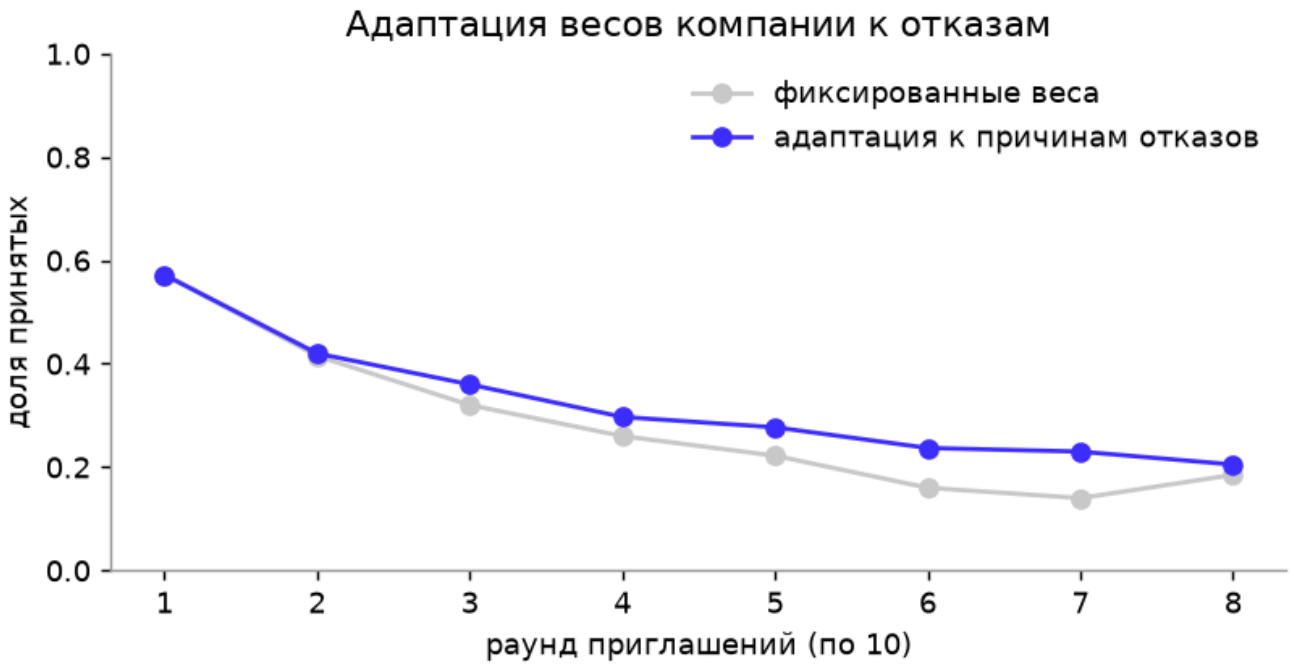
Чувствительность к определению релевантности. Пороги синтетической релевантности условны, поэтому метрики пересчитаны при строгом (без запаса по навыкам, грейд строго равен, зарплата не выше вилки) и мягком (достаточно половины обязательных навыков, зарплата до +20%) определениях. nDCG@10:

Система	Базовое	Строгое	Мягкое
Ключевые слова (как hh)	0.486	0.354	0.508
BM25 по тексту резюме	0.296	0.228	0.306
Наш скоринг без теста (самооценка)	0.738	0.561	0.741
Наш скоринг без ФСП	0.759	0.670	0.762
Наш скоринг (полный)	0.760	0.665	0.763

Выводы сохраняются при всех определениях: наш скоринг выше поиска по ключевым словам и BM25, а профиль, подтверждённый тестом, ранжирует лучше самооценки. Полный скоринг и «без ФСП» почти равны и могут меняться местами.

Адаптация весов к отказам. Каждая из 40 вакансий — компания, которая 8 раундов подряд приглашает по 10 лучших ещё не приглашённых кандидатов. У кандидатов скрытое правило: отказ, если вилка ниже ожиданий, не подходит формат, знакомо меньше 2/3 обязательного стека или позиция ниже их истинного уровня; иначе согласие с вероятностью 0,8. Отказ с причиной поднимает вес соответствующего компонента (`scoring.adapt_weights`, шаг +10%, потолок ×2). Контроль — те же компании с фиксированными весами и теми же случайными числами.

	Доля принятых за все раунды	Раунды 5–8
Фиксированные веса	28.4%	17.7%
Адаптация к отказам	32.5%	23.7%
Разница	+0.041 (95% ДИ +0.030…+0.052, значимо)	+0.061 (95% ДИ +0.040…+0.082, значимо)



7. Разбор текста потребности

Три набора описаний вакансий:

- **шаблоны** — 68 текстов: 8 ручных и вакансии генератора синтетики. Навыки там названы так же, как в справочнике, поэтому метрики близки к 1 и мало что доказывают;
- **разработка** — 40 реалистичных текстов (`validation/data/needs.json`): «вода», опечатки, английский и смешанный язык, жаргон и транслитерация («постгрес», «кубер», «джангу»), вилки в разных форматах. По ним мы находили ошибки правил и дорабатывали их. До доработки на этом наборе было F1 по навыкам 0,865, грейд 75%, вилка 72,5%;
- **отложенный** — 20 текстов (`validation/data/needs_holdout.json`), написанных *после* того, как правила были заморожены. Это честная оценка на новых текстах.

Метрика	Шаблоны	Разработка	Отложенный	Разработка + LLM	Отложенный + LLM
Навыки: точность	1.000	0.982	0.958	0.949	0.924
Навыки: полнота	1.000	0.988	0.872	0.988	0.936
Навыки: F1	1.000	0.985	0.913	0.968	0.930
Навык в верном списке (обязательный / желательный)	99.4%	97.0%	100.0%	96.4%	100.0%
Специализация	100.0%	97.5%	100.0%	100.0%	100.0%
Грейд (где указан)	100.0%	100.0%	87.5%	100.0%	93.8%
Вилка	100.0%	100.0%	100.0%	95.0%	95.0%
Формат работы	100.0%	100.0%	100.0%	100.0%	100.0%

Столбцы «+ LLM»: правила, уточнённые языковой моделью (gigachat,local) с постпроверкой по справочнику.

Эмбеддинги проверены и не включены. Мы проверили локальную модель `paraphrase-multilingual-MiniLM-L12-v2` (fastembed, ONNX) для сопоставления незнакомых терминов со справочником и для текстовой близости «вакансия — профиль». На жаргоне она ошибается уверенно: «свифт» → Vue (косинус 0,97), «терраформ» → вёрстка (0,91), «кубер» → теорвер (0,68). На уровне текстов сигнал слабый: профиль про кредитный скоринг оказался ближе к вакансии платёжного шлюза (0,57), чем профиль про процессинг платежей (0,45). Выигрыш есть только на англоязычных текстах, а модель добавляет ~250 МБ к образу и ~300 МБ памяти. Воспроизведение: `uv run --with fastembed python -m validation.embeddings_probe`. Вместо этого правила разбирают русские окончания и жаргон, а смысловое уточнение делает LLM, если задан ключ.

8. Разметка пар «вакансия – кандидат» людьми

Синтетическая релевантность из раздела 6 задана правилами, поэтому её проверяют люди. Из того же тестового набора взяты 10 вакансий (по две на специализацию) и по 10 кандидатов на каждую. Пул собран так, чтобы не подыгрывать нашей системе: топ-3 нашего скоринга, топ-3 поиска по ключевым словам, топ-2 BM25, остальные случайные из специализации, в перемешанном порядке. Разметчик видит вакансию и профиль кандидата (стек, опыт, результат теста по навыкам, ФСП, ожидания) и

ставит оценку 0–3. **Оценок системы он не видит.** Форма: `docs/validation/labeling/index.html` (пересобирается командой `uv run python -m validation.labeling`), ответы: `docs/validation/labels/*.json`.

Набор версии `20331c687ad0`. Разметчиков: 3, каждый размечал независимо; размечено пар: 100 (10 вакансий). Оценка пары — среднее по разметчикам. Доля подходящих (оценка ≥ 2) в пуле — 35.0%. Системы сравниваются по тому, как они упорядочивают один и тот же пул.

Согласие разметчиков между собой. Взвешенная каппа -0.08 (по парам разметчиков: -0.21, +0.00, -0.02); оценки расходятся не больше чем на балл в 69.0% случаев.

Разметчик	«0»	«1»	«2»	«3»
Человек 1	16	20	29	31
Человек 2	10	40	35	13
Человек 3	9	35	38	18

Ограничение. Согласие на уровне случайности: разметчики по-разному понимают шкалу 0–3 (у них разная строгость, это видно по распределению оценок) и по-разному взвешивают опыт, стек, грейд и ожидания. Средняя оценка при таком согласии — шум, поэтому **выводов о качестве ранжирования по этой разметке мы не делаем**: таблицы ниже приведены для полноты, различия систем в них не значимы. Качество подбора подтверждено в разделе 6 на синтетической разметке. Следующий шаг — рубрика с примерами для каждой оценки и калибровка разметчиков на 10 общих парах до основной разметки; форма и пул для этого готовы.

Система	nDCG пула (95% ДИ)	P@3	ρ Спирмена со средней оценкой
Ключевые слова (как hh)	0.876 (0.83–0.92)	0.400	-0.028
BM25 по тексту резюме	0.851 (0.80–0.90)	0.367	-0.127
Наш скоринг без теста (самооценка)	0.849 (0.79–0.91)	0.333	-0.000
Наш скоринг без ФСП	0.835 (0.78–0.90)	0.400	0.009
Наш скоринг (полный)	0.857 (0.80–0.92)	0.400	0.047

Проверка синтетической разметки	Значение
Совпадение средней оценки разметчиков с синтетической релевантностью	26.0% (±1 балл — 73.0%)
Взвешенная каппа «разметчики — синтетика»	0.027

9. Пилот на людях

Синтетика проверяет механику, но не реальные параметры заданий. Поэтому тест проходят живые люди, после теста заполняют необязательную анкету (стаж, должность, самооценка, грейд по оценке тимлида) и могут пожаловаться на конкретное задание. Выгрузка обезличена (кандидат заменён псевдонимом): `GET /api/assessment/pilot/export` по токену `PILOT_EXPORT_TOKEN`, затем `uv run python -m validation.pilot --fetch <адрес стенда> --token <токен>`.

Данных пилота пока нет; раздел заполнится после выгрузки.

10. Разбор резюме (PDF)

Вымышленные резюме в пяти вёрстках, которые встречаются в реальных PDF (`validation/resumes.py`, файлы в `validation/data/resumes/`): одна колонка, две колонки с боковой панелью, выгрузка в стиле hh.ru с таблицей мест работы, английское резюме и резюме, где навыки описаны прозой. Форматы периодов, телефонов и ссылок на Telegram различаются намеренно. PDF разбирается тем же кодом, что в продукте (`resume.py`, PyMuPDF), без LLM.

На наборе разработки (15 резюме) мы нашли ошибки парсера и исправили их: периоды «03.2019 – н.в.» и «с 2016 по 2019», перенос «настоящее время» внутри ячейки таблицы, телефоны с 8, имя после боковой панели, ложный Telegram внутри слова «PostgreSQL». Отложенный набор (6 резюме) написан после исправлений, с форматами, под которые парсер не настраивался: фамилия заглавными с отчеством, «2019 г. – 2021 г.», «Mar 2020 – current», явная строка «Опыт работы — 7 лет 9 месяцев».

Поле	До исправлений (разработка)	После (разработка)	**Отложенный**
ФИО	80.0%	100.0%	83.3%
E-mail	100.0%	100.0%	100.0%
Телефон	73.3%	100.0%	100.0%
Telegram	80.0%	100.0%	100.0%
Город	100.0%	100.0%	100.0%
Стаж (±1 год)	33.3%	100.0%	66.7%
Специализация	100.0%	100.0%	100.0%
Навыки: F1	0.975	0.994	**0.984**

Ошибки на отложенном наборе: `holdout_01_plain.pdf: full_name`, `holdout_01_plain.pdf: experience`, `holdout_02_sidebar.pdf: experience`. Карточка после загрузки PDF — черновик: кандидат проверяет и подтверждает каждое поле, а при заданном ключе LLM дополняет то, что правила не нашли.

11. Нагрузка

`uv run python -m validation.load --url <стенд> --users 50`: 50 одновременных пользователей (70% работодателей, 30% кандидатов) 3 минуты работают со стендом из `docker compose` с демо-данными. Каждый делает паузу 0,5–2 с между действиями, как человек.

Прогон 2026-10-06 14:09 UTC: 6744 запросов за 182 с (37.1 в секунду), ошибок 0.00%.

Операция	Запросов	p50, мс	p95, мс	p99, мс	Ошибок
Подборка по потребности	1658	104	218	325	0
Поиск по банку с фильтрами	1452	70	143	236	0
Карточка кандидата	900	35	102	156	0
Вакансии с оценкой соответствия	716	135	376	1320	0
Кабинет кандидата	693	36	118	214	0
Кандидаты категории	449	77	135	404	0
Генерация теста по специализации	434	61	109	206	0
Профиль кандидата	407	28	85	152	0
Список потребностей	35	374	486	544	0

Первый прогон нашёл узкое место: список вакансий кандидата пересчитывал траст-скор компании для каждой вакансии (p95 4,8 с, p99 8,3 с). После кэширования траст-скора на компанию в пределах запроса — значения выше. Память всего стека под нагрузкой — около 1 ГБ (`docker stats`: backend ~300 МБ, Keycloak ~525 МБ, PostgreSQL ~85 МБ).

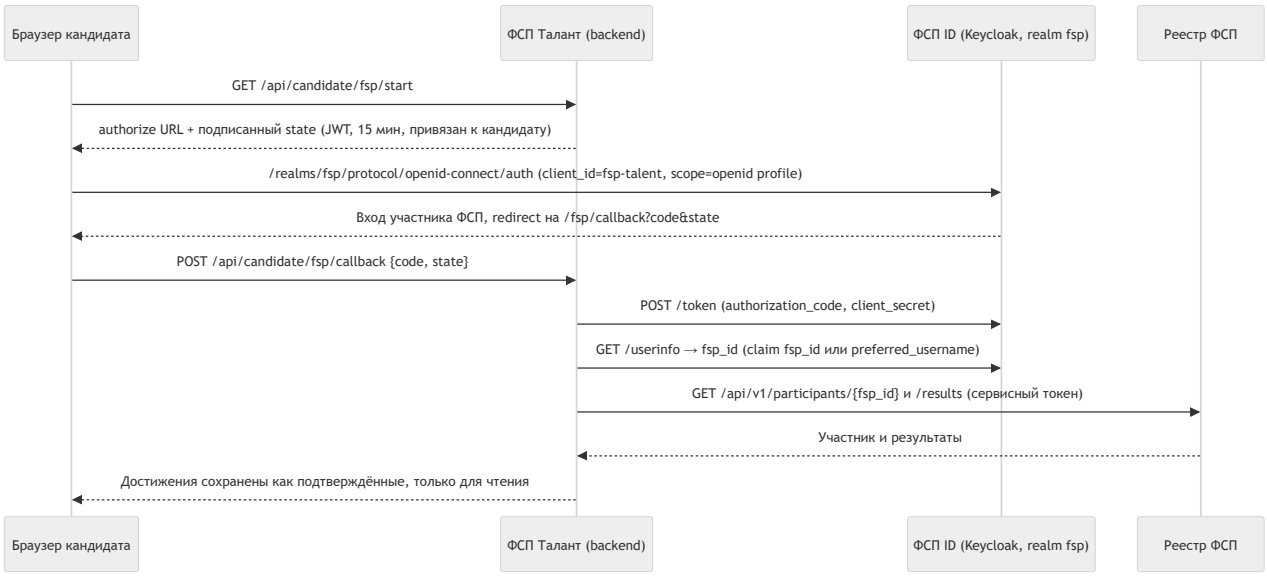
Ограничения и что дальше

- Синтетика проверяет корректность механики, но не реальные параметры заданий. После хакатона: пилот на 50–100 участниках ФСП, калибровка a и b по реальным ответам (маргинальная оценка максимального правдоподобия), сравнение грейда с экспертной оценкой тимлидов (корреляция Спирмена).

4. Схема интеграции с реестром ФСП

API и структура данных ФСП на хакатоне не предоставлялись. Ниже — **наше предложение** контракта, которое реализовано и работает в демо: Keycloak как ФСП ID и отдельный сервис-мок реестра.

Поток привязки ФСП ID



Свойства потока: - `client_secret` хранится только на backend. Параметр `state` защищает от CSRF и от подмены кандидата. - Один ФСП ID нельзя привязать к двум профилям (409). - Кандидат может обновить данные («Обновить») или отвязать ФСП ID. - **Если истории нет:** участник существует, но результатов нет, — профиль остаётся полноценным, вклад ФСП равен нулю (не штрафу). Работодатель видит нейтральную метку «нет данных ФСП». - **Если участник не найден:** возвращается 404 с понятным сообщением. - **Режим без Keycloak** (`FSP_OIDC_ENABLED=false`): встроенная страница-заглушка ФСП ID с тем же потоком `code` / `state` .

Для совместимости с будущим ФСП ID на Keycloak достаточно настроить три параметра: `FSP_OIDC_ISSUER` , `FSP_OIDC_CLIENT_ID` и `FSP_OIDC_CLIENT_SECRET` . Сервер реестра — `FSP_REGISTRY_URL` .

Предлагаемый контракт реестра

`GET /api/v1/participants/{fsp_id}` , авторизация сервис-сервис (в проде — client credentials в Keycloak):

```
{
  "fsp_id": "FSP-123457",
  "full_name": "Мария Иванова",
  "birth_year": 2001,
  "region": "Краснодарского края",
  "rank": "МС",
  "rating": 1903,
  "disciplines": ["product", "security"]
}
```

`GET /api/v1/participants/{fsp_id}/results`:

```
[{
  "id": "FSP-123457-R1",
  "event": {"id": "EV-3f2a1c", "name": "Всероссийский хакатон ФСП «Цифровой прорыв»",
            "level": "federal", "discipline": "product", "date": "2024-09-19", "city": "Казань"},
  "place": 3, "participants_count": 246, "result_type": "prize",
  "team": {"name": "Null Pointer", "role": "Аналитик", "size": 4},
  "certificate_url": "https://fsp-russia.com/certificates/FSP-123457-R1.pdf"
}]
```

Справочники:

Поле	Значения
`level`	`regional`, `federal`, `international`
`discipline`	`product`, `algorithmic`, `security`, `drones`, `robotics` — пять дисциплин спортивного программирования
`result_type`	`winner`, `prize`, `finalist`, `participant`
`rank`	спортивный разряд: `""`, `III`, `II`, `I`, `КМС`, `МС`, `МСМК`

Что и как показываем работодателю

- Бейдж «ФСП · разряд КМС» в карточке, список результатов: событие, уровень, дисциплина, место из N, роль в команде, год. Пометка «подтверждено реестром».
- Роль «Капитан» в команде автоматически добавляет кандидату роль «Капитан команды».
- Дисциплина отображается на навыки: алгоритмическое → алгоритмы; ИБ → security, Linux, HTTP; продуктовое → проектирование, REST, git.

Сигнал ФСП (`fsp/scoring.py`) — насыщающаяся сумма 1 − exp(−T), где

T = 0,7 · Σ_призовые w_i + min(0,3; 0,7 · Σ_участия w_i) + вес разряда, w_i = результат (победитель 1 / призёр 0,75 / финалист 0,45 / участник 0,2) × давность (полураспад 2 года).

Разряд даёт от 0,1 (III) до 1,0 (МСМК).

Принципы (по ответам постановщиков): - **Нет иерархии соревнований и дисциплин.** Региональный хакатон и всероссийский чемпионат, продуктовое и алгоритмическое программирование весят одинаково. Уровень и дисциплина показываются работодателю, но в скоринг не входят. - **Важны число соревнований и результат.** Победитель и призёр весят заметно больше, чем участник. - **Профиль нельзя набить онлайн-явками.** Суммарный вклад участий без места ограничен потолком 0,3. Это меньше одного призового места (0,7 · 0,75 ≈ 0,53). - **Вклад в командный результат не делим.** FSP ID не фиксирует, кто что сделал в команде, поэтому командное место засчитывается каждому участнику одинаково. - **Свежий результат весит больше** (полураспад 2 года). На старте платформы архива почти нет, поэтому эффект небольшой.

Сигнал используется: - **в силе профиля внутри категории** — вес 0,25: выше те, у кого есть подтверждённые достижения (Т3); - **в соответствии потребности** — вес 0,05: валидация показала, что для вопроса «подходит ли кандидат» тест уже измеряет ту же способность; - в фильтре «только с достижениями ФСП» и в требовании потребности `require_fsp`.

Кандидат может скрыть достижения от работодателей в настройках приватности.

Что предлагаем ФСП для реальной интеграции

1. Claim `fsp_id` в токене ФСП ID (mapper атрибута пользователя). Тогда не нужно полагаться на `preferred_username`.
2. Эндпоинты реестра, как выше, с пагинацией и `updated_since` для инкрементальной синхронизации.
3. Webhook `participant.result.created`, чтобы достижения появлялись в профиле без ручного «Обновить».
4. Ссылки на верифицируемые сертификаты (подпись или QR), которые мы показываем работодателю.
5. Согласие участника на передачу результатов платформе как OAuth-scope `fsp.results.read`.

5. Описание API

Полная спецификация OpenAPI 3 генерируется автоматически: - интерактивная документация — `/api/docs` (Swagger UI) и `/api/redoc`; - JSON — `/api/openapi.json`.

Каждый эндпоинт описан со схемами запросов и ответов и кодами ошибок.

Общие правила

- **Авторизация:** `Authorization: Bearer <access_token>`. Токен выдают `/api/auth/login` и `/api/auth/verify-email`.
- **Формат ошибки:**
- обычная ошибка — `{"detail": {"code": "<машинный код>", "message": "<текст для пользователя>"}}`;
- ошибка валидации — `422` с `{"detail": {"code": "validation_error", "message": "...", "errors": [...]}}`.
- **Коды ответов:**
- `401` — нет или истёк токен;
- `403` — роль не та;
- `404` — нет объекта или нет доступа к нему;
- `409` — конфликт состояния;
- `429` — лимит (квота приглашений, частота тестов).

Эндпоинты

auth

Метод	Путь	Назначение
POST	`/api/auth/register`	Регистрация (candidate / employer); письмо с подтверждением
POST	`/api/auth/verify-email`	Подтверждение email → токен
POST	`/api/auth/resend-verification`	Повторное письмо
POST	`/api/auth/login`	Вход (403 `email_not_verified`, если email не подтверждён)
POST	`/api/auth/demo-login?role=`	Вход под демо-аккаунтом: `employer`, `candidate`; `candidate_new` каждый раз создаёт нового кандидата с экспресс-тестом
GET	`/api/auth/me`	Текущий пользователь

candidate

Метод	Путь	Назначение
GET/PUT	`/api/candidate/profile`	Профиль
POST	`/api/candidate/onboarding`	Опрос: отрасли, специализация, стек, опыт, предполагаемый грейд

Метод	Путь	Назначение
PUT	`/api/candidate/privacy`	Видимость, «открыт к предложениям», порог зарплаты, что показывать
POST	`/api/candidate/consents`	Согласия на обработку и публикацию (152-ФЗ)
POST	`/api/candidate/resume`	PDF-резюме → распознанные поля (не сохраняются)
GET	`/api/candidate/profile.pdf`	Стандартизированный PDF-профиль с QR
DELETE	`/api/candidate/account`	Удаление аккаунта и данных
GET	`/api/candidate/fsp/start`	Начало привязки ФСП ID (URL авторизации)
POST	`/api/candidate/fsp/callback`	Обмен кода, загрузка достижений
POST/DELETE	`/api/candidate/fsp/refresh`, `/api/candidate/fsp`	Обновить / отвязать
GET	`/api/candidate/invitations`	Входящие приглашения (+ траст-скор компании)
POST	`/api/candidate/invitations/{id}/view|accept|decline|complain`	Действия с приглашением
GET	`/api/candidate/vacancies`	Вакансии с оценкой соответствия профилю
POST	`/api/candidate/vacancies/{id}/apply`	Отклик
GET	`/api/candidate/applications`	Мои отклики
POST	`/api/candidate/applications/{id}/withdraw`	Отозвать отклик
GET	`/api/candidate/tasks`	Регулярные задания (новое — раз в неделю)
POST	`/api/candidate/tasks/{id}/submit|skip`	Ответ или пропуск
GET	`/api/candidate/dashboard`	Сводка кабинета

assessment

Метод	Путь	Назначение
GET	`/api/assessment/eligibility`	Доступные грейды и ограничения по частоте
POST	`/api/assessment/sessions`	Начать тест (`target_grade`, опционально `vacancy_id`)
GET	`/api/assessment/sessions/{id}`	Текущее задание или результат с разбором
POST	`/api/assessment/sessions/{id}/answer`	Ответ (`item_id`, `response`, `signals`)
GET	`/api/assessment/history`	Попытки и история грейда
POST	`/api/assessment/blueprint-preview`	Публично: вакансия или текст → blueprint и примеры заданий
GET	`/api/assessment/bank`	Публично: банк позиций, параметры IRT, экспозиция, статус детектора

employer

Метод	Путь	Назначение
GET/PUT	`/api/employer/company`	Профиль компании, траст-скор, квота, вебхук
POST	`/api/employer/needs/parse`	Текст потребности → структура
GET/POST/PUT/DELETE	`/api/employer/needs[/{id}]`	Потребности (сохранённые подборки)
POST	`/api/employer/needs/{id}/shortlist`	Подборка: категории и кандидаты с объяснением (тело — фильтры)
POST	`/api/employer/needs/{id}/pin|hide`	Закрепить / скрыть кандидата
GET	`/api/employer/categories`	Карта категорий (число кандидатов, с ФСП)
POST	`/api/employer/categories/{spec}/{grade}`	Кандидаты категории по силе профиля
POST	`/api/employer/search`	Поиск по банку с фильтрами
GET	`/api/employer/candidates/{id}?need_id=`	Карточка кандидата (контакты — после принятия или отклика)
POST	`/api/employer/invitations/preview`	Попадает ли вилка в ожидания (без суммы), остаток квоты
POST	`/api/employer/invitations`	Отправить приглашение (вилка обязательна)
GET	`/api/employer/invitations`	Отправленные приглашения и статусы
POST	`/api/employer/invitations/{id}/withdraw`	Отозвать
GET/POST/PUT	`/api/employer/vacancies[/{id}]`	Вакансии
GET	`/api/employer/vacancies/{id}/applications`	Отклики с контактами, отсортированы по соответствию
POST	`/api/employer/applications/{id}/status`	Статус отклика

Метод	Путь	Назначение
GET/POST	`/api/employer/tasks`	Регулярные задания
GET	`/api/employer/tasks/{id}/submissions`	Решения кандидатов
GET	`/api/employer/analytics`	Воронка, причины отказов, текущие веса ранжирования

common

Метод	Путь	Назначение
GET	`/api/catalog`	Справочники
GET	`/api/public/verify/{code}`	Проверка подлинности PDF-профиля по QR
GET	`/api/public/stats`	Открытая статистика
GET/POST	`/api/notifications`, `/api/notifications/read`	Уведомления
GET	`/api/health`	Проверка работоспособности

Вебхуки для ATS

POST на `webhook_url` компании. Тело — `{"event": "...", "data": {...}}`. Заголовки: - `X-FSP-Talent-Event`; - `X-FSP-Talent-Signature` — HMAC-SHA256 тела, ключ `webhook_secret`.

События: - `invitation.sent`; - `invitation.accepted` — с контактами кандидата; - `invitation.declined` — с причиной; - `application.created`.

Журнал доставок хранится в `webhook_deliveries`.

Реестр ФСП (мок)

Отдельный сервис `fsp_registry`. Swagger — `/docs` внутри контейнера. Контракт описан в 04-fsp-integration.md.

6. За рамками MVP: защита от недобросовестных работодателей и интеграция с ATS

ТЗ ждёт здесь концепцию и архитектурный подход. Часть механизмов уже работает в MVP, они отмечены «реализовано».

Защита от фиктивных вакансий и недобросовестных работодателей

Угрозы

1. **Фиктивные вакансии:** сбор базы контактов, «завлекающие» вилки, мошенничество.
2. **Массовые нерелевантные рассылки.**
3. **Расхождение условий:** в приглашении одна вилка, на собеседовании другая.
4. **Компании-однодневки и чужие бренды.**

Многоуровневая модель

Уровень	Механизм	Статус
Данные кандидата	Контакты закрыты до согласия; точная сумма ожиданий не раскрывается никогда	реализовано
Частота	Квота 20 приглашений в неделю, обязательный персональный текст, одно активное приглашение на пару «компания–кандидат»	реализовано
Траст-скор	Контрольная сумма ИНН, совпадение домена почты и сайта, полнота профиля, возраст аккаунта, доля принятых приглашений, жалобы, аномальная вилка	реализовано (`trust.py`), виден кандидату
Аномалии	Вилка в 2,5 раза выше медианы рынка категории → сигнал «слишком хорошо, чтобы быть правдой»	реализовано
Обратная связь	Жалоба кандидата из приглашения: «фиктивная», «условия не совпали», «спам», «грубость»	реализовано, снижает траст-скор
Верификация юрлица	Запрос ЕГРЮЛ/ЕГРИП (ФНС, DaData): статус «действующая», дата регистрации, совпадение названия; подтверждение домена по DNS TXT или письмом на admin@	концепция
Модерация по риску	Низкий траст-скор → приглашения и вакансии уходят в очередь модерации, лимит квоты снижается до 5; высокий — без модерации	концепция
Пост-фактум	Опрос кандидата через 2 недели после принятия: «условия совпали?» → вес в траст-скоре; повторные нарушения → блокировка	концепция

Архитектура: `trust-service` подписан на события `invitation.*`, `complaint.created`, `vacancy.published` и пересчитывает скор инкрементально. Внешние проверки (ФНС, DNS) выполняются асинхронно воркером. Результаты кэшируются в `company_verifications`.

Интеграция с ATS работодателей

Цель: работодатель живёт в своей ATS (Хантфлоу, Talantix, E-Staff, Potok), а платформа служит источником подтверждённых кандидатов.

Направление	Механизм	Статус
Платформа → ATS	Вебхуки `invitation.sent/accepted/declined`, `application.created` с подписью HMAC; при принятии — контакты и профиль кандидата	реализовано
ATS → платформа	REST API с токеном компании: потребности, вакансии, приглашения, статусы; OpenAPI позволяет сгенерировать клиента	API реализован, токены компании — концепция
Формат кандидата	Экспорт в JSON Resume / HR-XML + ссылка на PDF-профиль с QR-проверкой	PDF реализован, экспорт — концепция
Готовые коннекторы	Адаптеры для популярных ATS: маппинг этапов воронки ATS на статусы приглашений, двусторонняя синхронизация	концепция
SSO для HR	Вход сотрудников работодателя через корпоративный IdP (OIDC/SAML) на той же Keycloak-инфраструктуре	концепция

Принцип: персональные данные уходят в ATS **только после согласия кандидата**, то есть после принятия приглашения или собственного отклика. Это те же правила видимости, что и в интерфейсе.

7. Используемые библиотеки и компоненты

Весь стек — открытое ПО под лицензиями MIT, Apache-2.0, BSD, PostgreSQL License, OFL. Версии зафиксированы: - backend — `backend/uv.lock`; - frontend — `frontend/package-lock.json`; - образы — `docker-compose.yml` и Dockerfile.

Инфраструктура

Компонент	Версия	Лицензия	Назначение
PostgreSQL	16 (alpine)	PostgreSQL	Основная СУБД
Keycloak	26.1	Apache-2.0	ФСП ID: OIDC-провайдер, realm `fsp`
nginx	1.27 (alpine)	BSD-2	Раздача SPA, обратный прокси `/api`, `/auth`, `/mail`
Mailpit	1.21	MIT	SMTP и веб-ящик для писем подтверждения на стенде
Python	3.12	PSF	Backend
Node.js	20 (сборка)	MIT	Сборка frontend

Backend (Python)

Библиотека	Версия	Лицензия	Назначение
fastapi	0.142.2	MIT	HTTP API, OpenAPI
uvicorn[standard]	0.54.0	BSD-3	ASGI-сервер
sqlalchemy	2.1.3	MIT	ORM
alembic	1.20.0	MIT	Миграции
psycopg[binary]	3.3.6	LGPL-3	Драйвер PostgreSQL
pydantic	2.13.5	MIT	Схемы и валидация
pydantic-settings	2.15.0	MIT	Конфигурация из env
pyjwt	2.15.1	MIT	JWT-токены
argon2-cffi	25.1.0	MIT	Хеширование паролей (argon2id)
email-validator	2.3.0	Unlicense	Валидация email
python-multipart	0.0.32	Apache-2.0	Загрузка файлов
httpx	0.28.1	BSD-3	OIDC, реестр ФСП, вебхуки
numpy	2.5.3	BSD-3	IRT: EAP, информация Фишера
pymupdf	1.28.2	AGPL-3 / коммерческая	Извлечение текста из PDF-резюме*
reportlab	5.0.1	BSD	Генерация PDF-профиля
qrcode	8.2	BSD	QR-код проверки профиля
matplotlib (dev)	3.11.2	PSF-based	Графики отчёта валидации

Библиотека	Версия	Лицензия	Назначение
pytest (dev)	9.1.1	MIT	Тесты
@playwright/test (e2e)	1.63.0	Apache-2.0	Сквозной сценарий в браузере, скриншоты (`e2e/`)
fastembed (не зависимость)	0.8.1	Apache-2.0	Только для воспроизведения проверки эмбеддингов: `uv run --with fastembed python -m validation.embeddings_probe`
ruff (dev)	0.16	MIT	Линтер

* PyMuPDF распространяется под AGPL. Для коммерческой эксплуатации замена — `pypdf` (BSD): модуль `resume.py` использует только извлечение текста, это одна функция `pdf_to_text`.

Frontend

Библиотека	Версия	Лицензия	Назначение
react, react-dom	19.3.0	MIT	UI
react-router	8.4.0	MIT	Маршрутизация
@tanstack/react-query	5.104.1	MIT	Запросы и кэш
tailwindcss, @tailwindcss/vite	4.3.3	MIT	Стили
lucide-react	1.52.0	ISC	Иконки
recharts	3.10.1	MIT	Графики аналитики
prism-react-renderer	2.4.1	MIT	Подсветка кода в заданиях
clsx	2.1.1	MIT	Утилита классов
@fontsource-variable/golos-text	5.3.0	OFL-1.1	Шрифт интерфейса
@fontsource-variable/jetbrains-mono	5.3.0	OFL-1.1	Моноширинный шрифт (брендбук ФСП)
typescript	6.0.3	Apache-2.0	Типизация
vite	8.3.2	MIT	Сборка

Шрифты в PDF

PT Sans (OFL-1.1, ParaType) и JetBrains Mono (OFL-1.1) лежат в `backend/app/assets/fonts` вместе с текстами лицензий.

Внешние сервисы

Обязательных нет: решение работает полностью офлайн, NLP и оценка заданий детерминированны.

Необязательно подключается отечественная языковая модель (`LLM_PROVIDER`): **GigaChat** (Сбер, REST API `/api/v1/chat/completions`, OAuth по ключу авторизации) или **YandexGPT** (Yandex Cloud Foundation Models, `/foundationModels/v1/completion`), либо открытая модель на собственном сервере: **Ollama** 0.40.0 (MIT) с **Qwen3-4B-Instruct-2507** (Apache-2.0) через OpenAI-совместимый API — резюме не

покидают сервер (152-ФЗ). Модели выстраиваются в цепочку (`gigachat,local`): при сбое сети следующая подхватывает запрос. Обращение идёт через `httpx`, без SDK. Модель уточняет разбор потребности и резюме и проверку ответов «опишите подход». Правильные ответы теста она не формирует и не проверяет.

8. Сборка, развёртывание и локальный запуск

Вариант 1. Docker Compose (рекомендуется)

Требования: Docker 24+ с Compose v2, 3 ГБ свободной памяти, свободный порт 8080. ОС любая: Linux, macOS или Windows с Docker Desktop. GPU не нужен: локальных моделей нет, языковая модель необязательна и вызывается по API. Стек с запасом укладывается в ориентир стенда жюри (8 CPU, 32–64 ГБ RAM).

Для проверки: на странице входа кнопка «Новый кандидат» создаёт свежий аккаунт на каждый клик, тест в нём идёт в экспресс-режиме (3–5 минут). «Демо-работодатель» открывает кабинет с готовыми потребностями и подборками.

```
git clone <repo> fsp-talent
cd fsp-talent
docker compose up -d --build
```

При первом запуске: 1. PostgreSQL создаёт базу. 2. Backend применяет миграции Alembic (`alembic upgrade head`). 3. Backend загружает демо-данные (`SEED_DEMO=true`). Это идемпотентно: повторный запуск ничего не дублирует. 4. Keycloak импортирует realm `fsp` из `deploy/keycloak/realm-fsp.json`.

Готовность проверяется так:

```
docker compose ps
curl http://localhost:8080/api/health   # {"ok": true, ...}
```

Адреса: приложение — http://localhost:8080, Swagger — `/api/docs`, ФСП ID — `/auth`, почта — `/mail`.

Полезные команды

```
docker compose logs -f backend                    # логи API
docker compose down                               # остановить
docker compose down -v                            # остановить и удалить данные (демо
пересоздастся)
docker compose exec backend python -m app.seed.run --reset   # пересоздать демо-данные
```

Переменные окружения

Задаются в `.env` рядом с `docker-compose.yml`.

Переменная	По умолчанию	Назначение
`PUBLIC_URL`	`http://localhost:8080`	Публичный адрес: ссылки в письмах, QR, OIDC redirect, Keycloak hostname
`HTTP_PORT`	`8080`	Внешний порт nginx
`JWT_SECRET`	dev-значение	Секрет подписи токенов — **сменить в проде**
`POSTGRES_PASSWORD`	`fsp`	Пароль БД

Переменная	По умолчанию	Назначение
`KEYCLOAK_ADMIN_PASSWORD`	`admin`	Пароль админки Keycloak
`SEED_DEMO`	`true`	Загрузить демо-данные при старте
`EXPOSE_EMAIL_LINKS`	`true`	Возвращать ссылку подтверждения в ответе API (только для демо)
`FSP_OIDC_ENABLED`	`true`	`false` — встроенная заглушка ФСП ID вместо Keycloak
`LLM_PROVIDER`	`none`	Языковая модель поверх правил NLP: `none`, `gigachat`, `yandex`, `local` или цепочка через запятую (`gigachat,local`: при сбое первой отвечает следующая, при сбое всех — правила)
`LLM_API_KEY`	—	GigaChat: ключ авторизации (Base64) из личного кабинета; YandexGPT: API-ключ сервисного аккаунта
`LLM_MODEL`	—	По умолчанию `GigaChat` или `yandexgpt-lite/latest`
`LLM_SCOPE`	`GIGACHAT_API_PERS`	GigaChat: `GIGACHAT_API_PERS`, `GIGACHAT_API_B2B` или `GIGACHAT_API_CORP`
`LLM_FOLDER_ID`	—	YandexGPT: идентификатор каталога Yandex Cloud
`LLM_VERIFY_SSL`, `LLM_CA_BUNDLE`	`true`, `certs/russian_trusted_ca.pem`	GigaChat использует сертификат НУЦ Минцифры: корневой и промежуточный сертификаты лежат в `backend/certs/` и встроены в образ
`LLM_TIMEOUT`	`30`	Таймаут ответа облачной модели, с; соединение повторяется до 3 раз
`LOCAL_LLM_URL`	`http://ollama:11434/v1`	Локальная открытая модель: OpenAI-совместимый API (Ollama, vLLM, llama.cpp server)
`LOCAL_LLM_MODEL`	`qwen3:4b-instruct`	Qwen3-4B-Instruct-2507 (Apache-2.0), ~2.5 ГБ; Ollama скачивает её при первом старте

Локальная модель запускается профилем: `docker compose --profile local-llm up -d` (+~3 ГБ RAM, без видеокарты ответ 10–40 с). Данные кандидатов при этом не покидают сервер.

Без ключа все сценарии работают на правилах. Проверить ключ: `cd backend && uv run python -m app.modules.llm "текст вакансии"` — команда покажет провайдера, разбор правилами и уточнённый разбор.

Вариант 2. Развёртывание на сервере с HTTPS

1. VPS с 2 vCPU и 4 ГБ RAM, Docker, домен с A-записью на сервер.
2. В `.env` задайте: `PUBLIC_URL=https://talent.example.ru`, `HTTP_PORT=8080`, `JWT_SECRET=<случайные 64 символа>`, `EXPOSE_EMAIL_LINKS=false` (если настроен настоящий SMTP) или `true` (для демо-стенда).

3. Поставьте перед стеком Caddy для автоматического TLS (`deploy/Caddyfile`):

`talent.example.ru { reverse_proxy localhost:8080 }`

`bash docker compose up -d --build caddy run --config deploy/Caddyfile # или caddy как systemd-сервис`

1. Nginx внутри стека передаёт `X-Forwarded-Proto`, Keycloak работает за прокси (`KC_PROXY_HEADERS=xforwarded`). Ссылки авторизации ФСП ID строятся от `PUBLIC_URL`.

Вариант 3. Локальная разработка без Docker

Backend (нужен uv). По умолчанию используется SQLite, ФСП ID работает в режиме заглушки, мок реестра вызывается в процессе:

```
cd backend
uv sync
uv run python -m app.seed.run --reset          # демо-данные в dev.db
SMTP_HOST= uv run uvicorn app.main:app --reload --port 8000
```

Frontend:

```
cd frontend
npm ci
npm run dev                                     # http://localhost:5173, /api проксируется на
:8000
```

Проверки

```
cd backend
uv run pytest -q                       # 341 тест: сквозной сценарий API, права доступа, политики,
уникальность
                                       # каждой позиции банка, CAT и экспозиция, скоринг, LLM-
слой, пилот, резюме
uv run ruff check app validation tests # линтер
uv run python -m validation.run        # валидация: docs/validation/report.md (~15 мин; --quick —
~5 мин)
uv run python -m validation.labeling   # форма разметки пар людьми:
docs/validation/labeling/index.html
uv run python -m validation.resumes    # пересобрать набор PDF-резюме для раздела 10
cd ../frontend && npm run build        # проверка типов и сборка
```

Против запущенного стенда (`docker compose up -d --build`):

```
cd e2e && npm ci && npx playwright install chromium
npx playwright test                    # сквозной сценарий ТЗ 3.3 + скриншоты всех страниц на 1440
и 375 px
                                       # (e2e/screens/), проверка отсутствия горизонтальной
прокрутки
cd ../backend
uv run python -m validation.load --users 50 --seconds 180   # нагрузка: p50/p95/p99 →
docs/validation/load.json
```

Результаты последнего нагрузочного прогона (50 одновременных пользователей, 3 минуты, ноутбук): 37 запросов в секунду без ошибок; p95 всех операций ≤ 380 мс (подборка — 218 мс, поиск — 143 мс, карточка кандидата — 102 мс). Память стека под нагрузкой — около 1 ГБ: backend ~300 МБ, Keycloak ~525 МБ, PostgreSQL ~85 МБ.

Пилот на людях

1. Задайте токен выгрузки: `PILOT_EXPORT_TOKEN=<случайная строка>` в `.env` рядом с `docker-compose.yml`, затем `docker compose up -d`.
2. Участники регистрируются, проходят тест и заполняют анкету «Помогите проверить тест» на странице результата.
3. `cd backend && uv run python -m validation.pilot --fetch http://<стенд> --token <токен>` сохраняет обезличенную выгрузку в `docs/validation/pilot/`; `uv run python -m validation.run` добавит раздел «Пилот» в отчёт.